全国计算机技术与软件专业技术资格（水平）考试

2020 年下半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1．在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。

2．在答题纸的指定位置填写准考证号、出生年月日和姓名。

3．答题纸上除填写上述内容外只能写解答。

4． 本试卷共 6 题，试题一至试题四是必答题，试题五至试题六选答 1 道。每题 15 分，满分 75 分。

5．解答时字迹务必清楚，字迹不清时，将不评分。

6．仿照下面的例题，将解答写在答题纸的对应栏内。

例题

2020 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是（1）月（2）日。

因为正确的解答是“11 月 7 日”，故在答题纸的对应栏内写上“11”和“7”（参看下表）。

例题	解答栏
（1）	11
（2）	7

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某工厂制造企业开发了智能检测系统以有效提升检测效率，节约人力资源，该系统的主要功能包括：

（1）基础信息管理。管理员对检测标准和监控规则等基础信息设置。

（2）检测模型部署。管理员对常用机器学习方法建立检测模型分布。

（3）图像采集。实时将检测多样的产品待检测建分存储，包括产品结构，生产时间，图像信号和产品图像。

（4）缺陷检测。根据检测模型和检测质量标准对图像采集所收到的产品检测信息中所有图像进行检测或所有图像检测合格。若一个产品出现一张图像检测不合格，就表示该产品不合格，对不合格产品，其检测结果包括，产品型号和不合格类型。

（5）质量检测。根据监控规则对产品质量进行监控，将检测情况展示给检测业务员，若不满足条件，向检测业务员发送质量报警，检测是质量发起远程控制部分，向检测设备发送控制指令进行处理。

（6）模型监控。在系统中部署的模型、产品的检测信息结合基础信 息进行监测分析，将模型运行情况发给监控人员。

现采用结构化方法对智能检测系统，进行分析与设计，获得如图 1-1 的上下文数据流图和图 1-2 的数据流图。

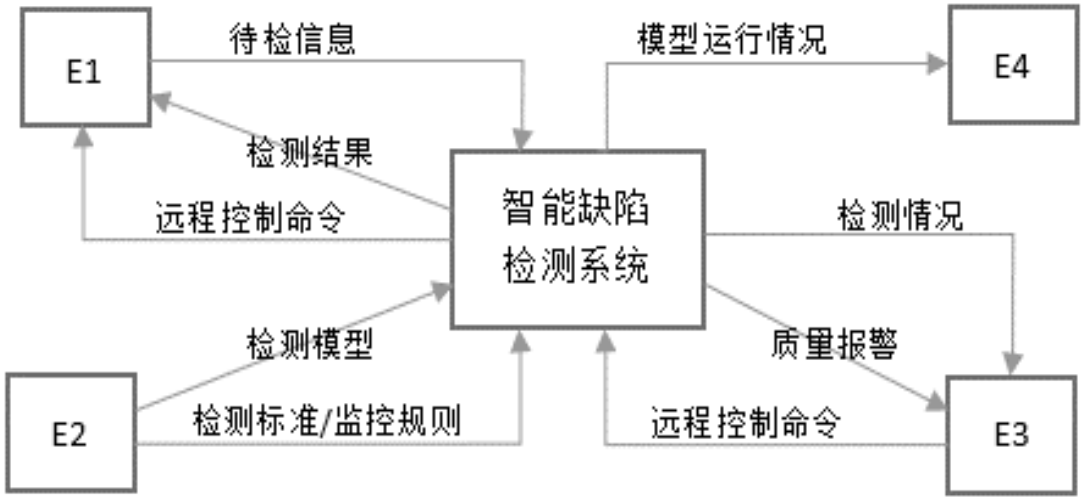


图 1-1

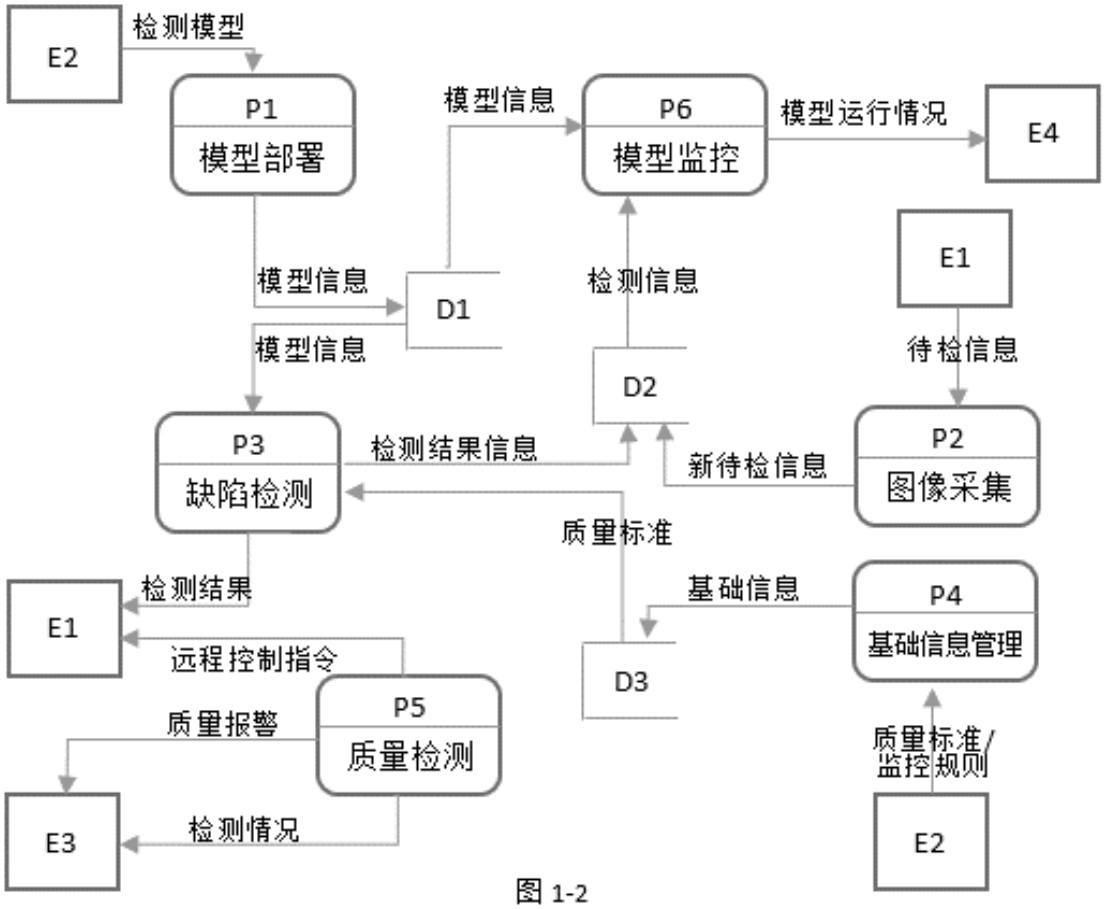


图 1-2

【问题 1】（4 分）

使用说明中的语句给出图 1-1 中的实体 E1～E4 的名称。

【问题 2】（3 分）

使用说明中的语句给出图 1-2 中的数据存储 D1～D3 的名称。

【问题 3】（5 分）

根据说明和图中术语，补齐图 1-2 中缺失的数据及起点和终点。

【问题 4】（3 分）

根据说明，采用结构化语言对缺陷检测的加工逻辑进行描述。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

M 集团拥有多个分公司，为了方便集团公司对各个分公司职员进行有效管理，集团公司决定构建一个信息平台以满足公司各项业 务管理需求。

【需求分析结果】

（1）分公司关系模式需要记录的信息包括分公司编号、名称、经理号、联系地址和电话。分公司编号唯一标记分公司关系模式中的每一个元组，每个分公司各有一名经理，负责分公司的管理工作，每个分公司设立仅为本分公司服务的多个业务部，业务部包括：研发部、财务部、采购部、交易部等。

（2）业务部关系模式需要记录的信息包括业务部编号、名称、地址、电话和分公司编号。业务部编号唯一标记业务部关系模式中的每 一个元组，每个业务部各有一名主管负责业务部的管理工作，每个业务部有多名职员，每个职员只能来源于一个业务部。

（3）职员关系模式需要记录的信息包括职员号、姓名、所属业务部编号、岗位、电话、家庭成员姓名和成员关系。其中岗位包括：经理、主管、研发员、业务员等。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

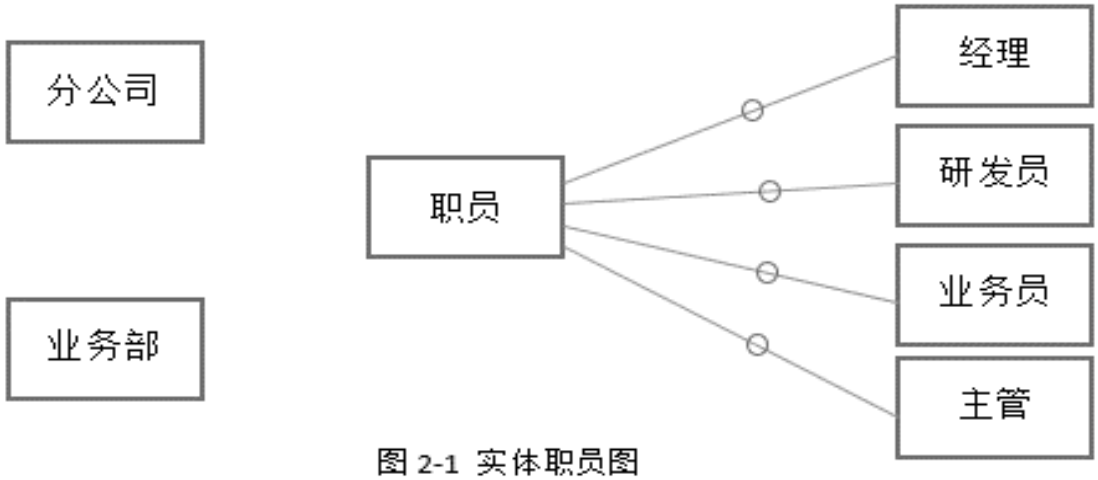


图 2-1 实体职员图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

分公司（分公司编号，名称，__（a）__，联系地址）

业务部（业务部编号，名称，__（b）__，电话），

职员（职员号，姓名，岗位，__（c）__，电话，家庭成员姓名，成员关系）

【问题 1】（4 分）

根据问题描述，补充 4 个联系，完善图 2-1 的实体联系图，联系名可用联系 1、联系 2、联系 3 和联系 4 代替，联系的类型为 1:1、1:n 和 m:n（或 1:1、1:*和*:*）

【问题 2】（3 分）

根据题意将以上关系模式中的空（a）～（c）的属性补充完整，并填入对应位置。

【问题 3】（4 分）

（1）分析分公司关系模式的主键和外键

（2）分析业务部关系模式的主键和外键

【问题 4】（4 分）

在职员关系模式中，假设每个职员有多名家属成员，那么职员关系模式存在什么问题？应如何解决？

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某房产公司欲开发一个房产信息管理系统，其主要功能描述如下：

（1）公司销售的房产（Property）分为住宅（House）和公寓（Cando）两类。针对每套房产，系统存储房产证明、地址、建造年份、建筑面积，销售报价、房产照片以及销售状态（在售，售出，停售）等信息。对于住宅，还需存储楼层、公摊面积、是否有地下室等信息；对于公寓，还需存储是否有阳台等信息。

（2）公司雇佣了多名房产经纪（Agent），负责销售房产，系统中需要存储房产经纪的基本信息，包括：姓名、家庭住址、联系电话、受雇的起止时间等。一套房产同一时段仅由一名房产经纪负责销售，系统中会记录房产经纪负责每套房产的起始时间和终止时间。

（3）系统用户（User）包括房产经纪和系统管理员（Manager），用户需经过系统身份验证之后才能登录系统。房产经纪登录系统之后，可以录入负责销售的房产信息，也可以查询所负责的房产信息。房产经纪可以修改其负责的房产信息，但需要经过系统管理员的审批授权。

（4）系统管理员可以从系统中导出所有房产的信息列表，系统管理员定期将售出和停售的房产信息进行归档，若公司确定不再销售某套房产，系统管理员将该房产信息从系统中删除。

现采用面向对象方法开发该系统，得到如图 3-1 所示的用例图和图 3-2 所示的初始类图。

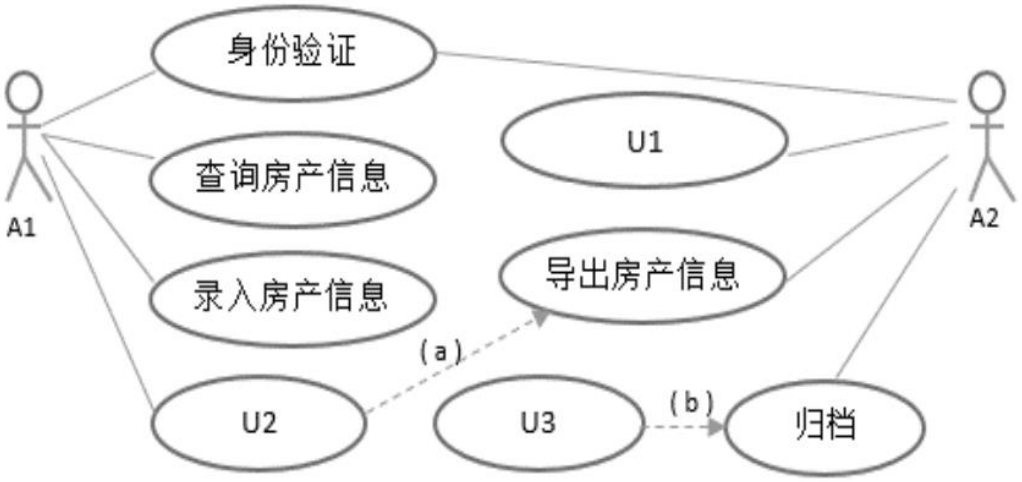


图3-1 用例图

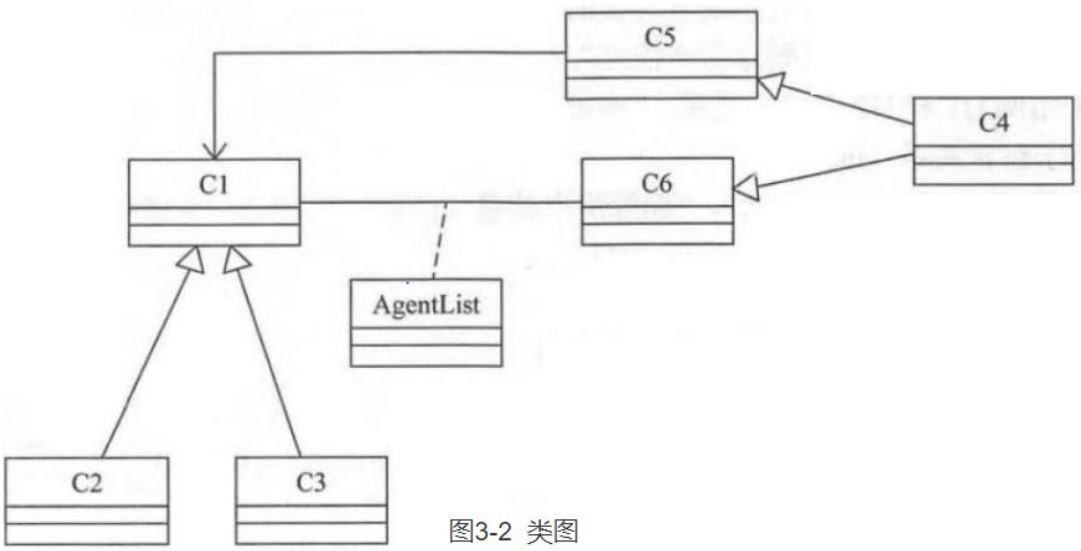


图3-2 类图

【问题 1】（7 分）

（1）根据说明中描述，分别给出图 3-1 中 A1 到 A2 所对应的参与者名称以及 U1 到 U3 所对应的用例名称。

（2）根据说明中描述，分别给图 3-1 中（a）和（b）用例之间的关系。

【问题 2】（6 分）

根据说明中描述，分别给图 3-2 中 C1～C6 所对应的类名称。

【问题 3】（2 分）

图 3-2 中 AgentList 是一个英文名称，用来进一步阐述 C1 和 C6 之间的关系，根据说明中的描述，给出 AgentList 的主要属性。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

希尔排序算法又称最小增量排序算法，其基本思想是：

步骤 1：构造一个步长序列 delta1，delta2，....，deltak，其中 delta1 = n / 2，后面的每个 delta 是前一个的 1 / 2，deltak = 1；

步骤 2：根据步长序列进行 k 趟排序；

步骤 3：对第 i 趟排序，根据对应的步长 delta，将等步长位置元素分，对同一组内元素在原位置上进行直接插入排序。

【C 代码】

下面的算法用 C 语言实现

（1）常量和变量说明

Data：待排序数组 data，长度为 n，待排序数据在 data[0]，data[1]，data[2]...，data[n - 1]中。

N：数组 data 中的元素个数

delta：步长数组

（2）C 程序

```
#include <stdlib.h>

void ShellSort(int data[], int n) {
    int* delta, k, i, t, dk, j;
    k = n;
    delta = (int*) malloc(sizeof(int) * (n / 2));
    i = 0;
    do {
        __(1)__;
        delta[i ++ ] = k;
    } while __(2)__;
```

```
    i = 0;
    while (  (3)  ) {
        for (k = delta[i]; k < n; ++ k )
            if (data[k] < data[k - dk]) {
                t = data[k];
                for (j = k - dk; j >= 0 && t < data[j]; j -= dk)
                    data[j + dk] = data[j];
                 (4) ;
            }
        ++ i;
    }
}
```

【问题 1】（8 分）

根据说明和 C 代码，填充 C 代码中的空（1）～（4）。

【问题 2】（4 分）

根据说明和 C 代码，该算法的时间复杂度__（5）__ $O(n^2)$（填写小于、等于或大于）。

该算法是否稳定__（6）__（是或否）。

【问题 3】（3 分）

对数组(15，9，7，8，20，-1，4)用希尔排序方法进行排序，经过第一趟排后得到的数组为__（7）__。

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入（n）处的字句写在答题纸的对应栏内。

【说明】

在线支付是电子商务的一个重要环节，不同的电子商务平台提供了不同的支付接口。现在需要整合不同电子商务平台的支付接口，使得客户在不同平台上购物时，不需要关心具体的支付接口。拟采用中介者（Mediator）设计模式来实现该需求，所设计的类图如图 6-1 所示。

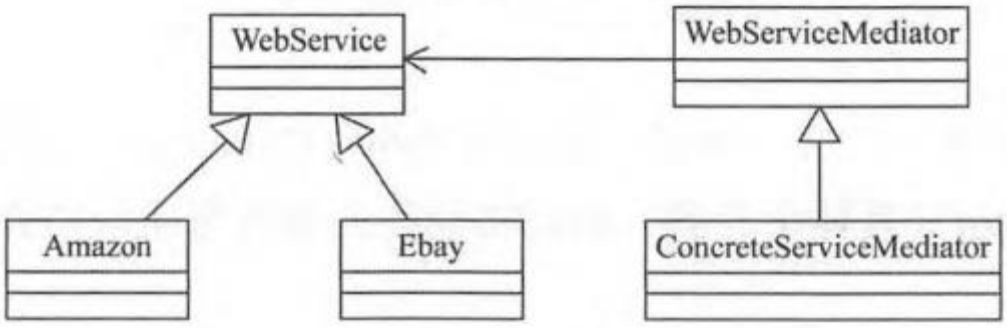


图 6-1 类图

【Java 代码】

```
import java.util.*;

interface WebServiceMediator {
    public  (1) ;
    public void SetAmazon(WebService amazon);
    public void SetEbay(WebService ebay);
}

abstract class WebService {
    protected  (2)  mediator;
    public abstract void SetMediator(WebServiceMediator mediator);
    public  (3) ;
    public abstract void search(double money);
}
```

```
class ConcreteServiceMediator implements WebServiceMediator {
    private WebService amazon;
    private WebService ebay;

    public ConcreteServiceMediator() {
        amazon = null;
        ebay = null;
    }

    public void SetAmazon(WebService amazon) {
        this.amazon = amazon;
    }

    public void SetEbay(WebService ebay) {
        this.ebay = ebay;
    }

    public void buy(double money, WebService service) {
        if (service == amazon)
            amazon.search(money);
        else
            ebay.search(money);
    }
}
```

```
class Amazon extends WebService {
    public void SetMediator(WebServiceMediator mediator) {
        this.mediator = mediator;
    }

    public void buyService(double money) {
        __(4)__;
    }

    public void search(double money) {
        System.out.println("Amazon receive: " + money);
    }
}

class Ebay extends WebService {
    public void SetMediator(WebServiceMediator mediator) {
        this.mediator = mediator;
    }

    public void buyService(double money) {
        __(5)__;
    }

    public void search(double money) {
        System.out.println("Ebay receive: " + money);
    }
}
```

全国计算机技术与软件专业技术资格（水平）考试

2021 年上半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1．在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。

2．在答题纸的指定位置填写准考证号、出生年月日和姓名。

3．答题纸上除填写上述内容外只能写解答。

4．本试卷共 6 道题，试题一至试题四是必答题，试题五至试题六选答 1 道。每题 15 分，满分 75 分。

5．解答时字迹务必清楚，字迹不清时，将不评分。

6．仿照下面的例题，将解答写在答题纸的对应栏内。

例题

2021 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是（1）月（2）日。

因为正确的解答是“5 月 29 日”，故在答题纸的对应栏内写上“5”和“29”（参看下表）。

例题	解答栏
（1）	5
（2）	29

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某停车场运营方为了降低运营成本，减员增效，提供良好的停车体验，欲开发无人值守停车系统，该系统的主要功能是：

（1）信息维护。管理人员对车位（总数、空余车位数等）计费规则等基础信息进行设置。

（2）会员注册。车主提供手机号、车牌号等信息进行注册，提交充值信息（等级、绑定并授权支付系统进行充值或交费的支付账号）不同级别和充值额度享受不同停车折扣点。

（3）车牌识别。当车辆进入停车场时，若有（空余车位数大于 1），自动识别车牌号后进行道闸控制，当车主开车离开停车场时，识别车牌号，计费成功后，请求道闸控制。

（4）计费。更新车辆离场时间，根据计费规则计算出停车费用，若车主是会员，提示停车费用：若储存余额够本次停车费用，自动扣费，更新余额，若储值余额不足，自动使用授权缴费账号请求支付系统进行支付，获取支付状态。若非会员临时停车，提示停车费用，车主通过扫描费用信息中的支付码调用支付系统自助交费，获取支付状态。

（5）道闸控制。根据道闸控制请求向道闸控制系统发送放行指令和接收道闸执行状态。若道闸执行状态为正常放行时，对入场车辆，将车牌号及其入场时间信息存入停车记录，修改空余车位数；对出场车辆更新停车状态，修改空余车位数。当因道闸重置系统出现问题（断网断电或是故障为抬杠等情况），而无法在规定的时间内接收到其返回的执行状态正常放行时，系统向管理人员发送异常告警信息，之后管理人员安排故障排查处理，确保车辆有序出入停车场。

现采用结构化方法对无人值守停车系统进行分析与设计，获得如图 1-1 所示的上下文数据流图和图 1-2 所示的 0 层数据流图。

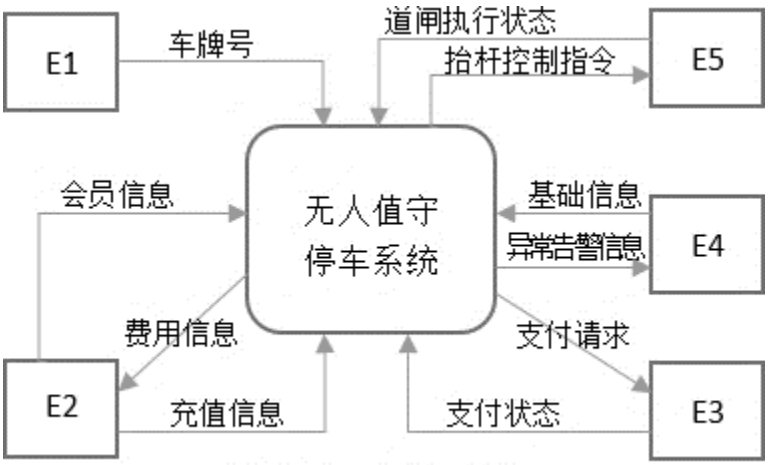


图 1-1 上下文数据流图

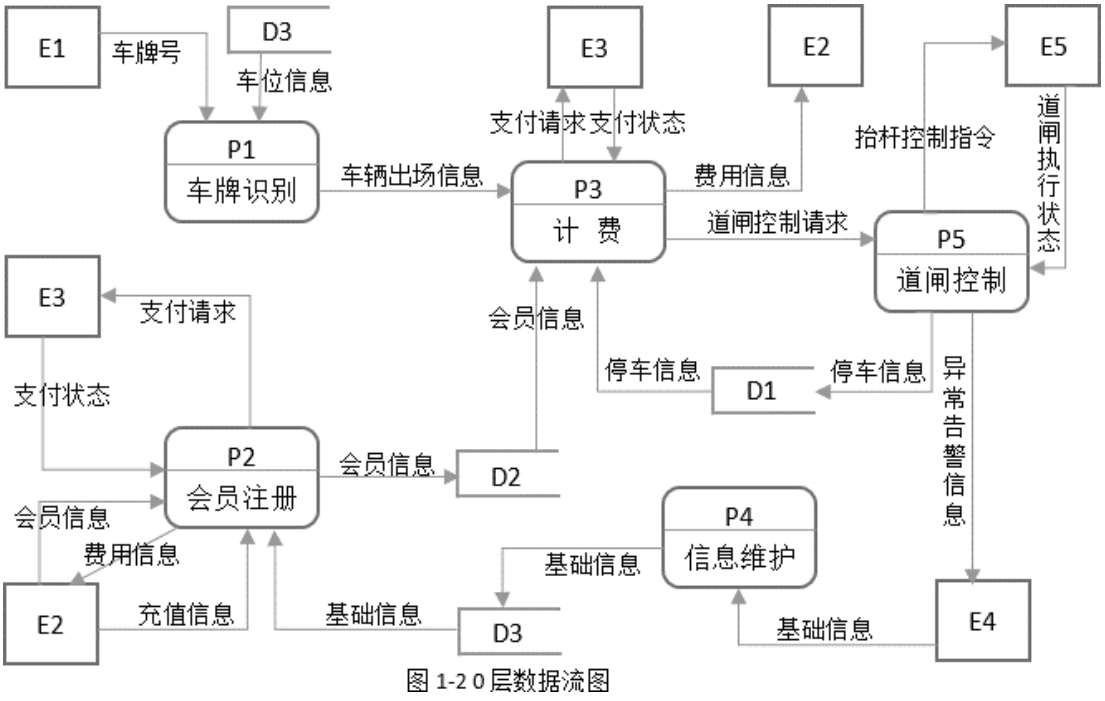


图 1-2 0 层数据流图

【问题 1】（5 分）

使用说明中的词语，给出图 1-1 中的实体 E1～E5 的名称。

【问题 2】（3 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1～D3 的名称。

【问题 3】（4 分）

根据说明和图中术语，补充图 1-2 中缺失的数据流及其起点和终点。

【问题 4】（3 分）

根据说明，采用结构化语言对“道闸控制”的加工逻辑进行描述。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某社区蔬菜团购网站，为规范商品收发流程，便于查询客户订单情况，需要开发个信息系统。请根据下述需求描述完成该系统的数据库设计。

【需求分析结果】

（1）记录蔬菜供应商的信息，包括供应商编号、地址和一个电话。

（2）记录社区团购点的信息，包括团购点编号、地址和一个电话。

（3）记录客户信息，包括客户姓名和一个电话。客户可以在不同的社区团购点下订单，不直接与蔬菜供应商发生联系。

（4）记录客户订单信息，包括订单编号、团购点编号、客户电话、订单内容和日期。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

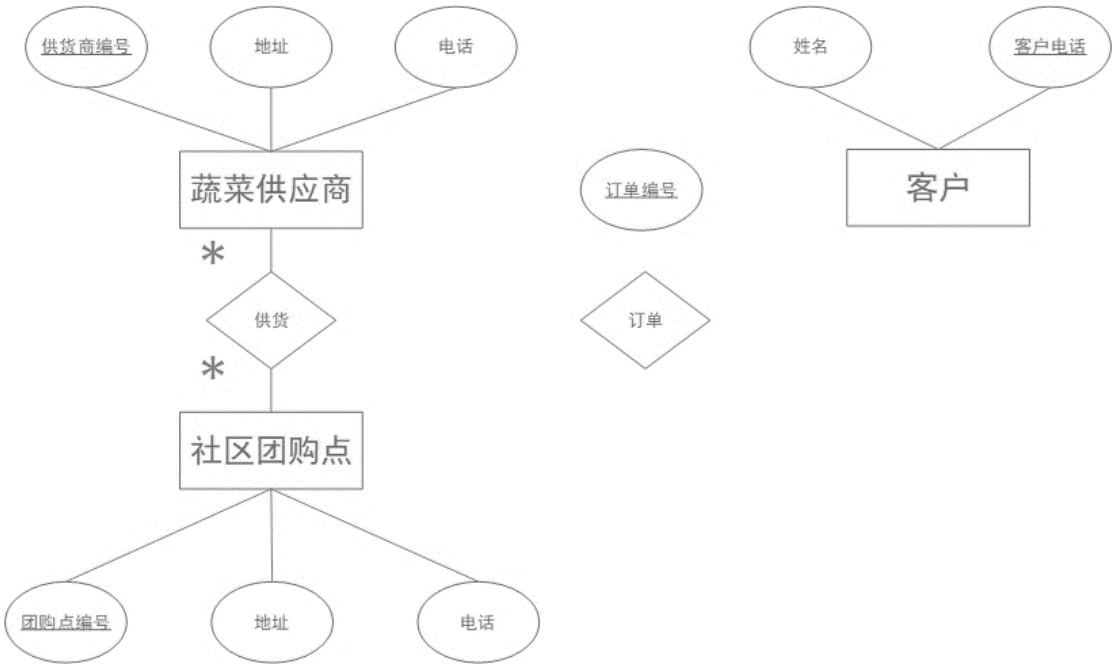


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

蔬菜供货商（供货商编号，地址，电话）

社区团购点（团购点编号，地址，电话）

供货（供货商编号，___（a）___）

客户（姓名，客户电话）

订单（订单编号，团购点编号，订单内容，日期，___（b）___）

【问题 1】（6 分）

根据问题描述，补充图 2-1 的实体联系图。

【问题 2】（4 分）

补充逻辑结构设计结果中的（a）、（b）两处空缺及完整性约束关系。

【问题 3】（5 分）

若社区蔬菜团购网站还兼有 代收快递的业务，请增加新的“快递”实体，并给出客户实体和快递实体之间的“收取’联系，对图 2-1 进行补充。“快递”关系模式包括快递编号、客户电话和日期。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某中医医院拟开发一套线上抓药 APP，允许患者凭借该医院医生开具的处方线上抓药，并提供免费送药上门服务。该系统的主要功能描述如下：

（1）注册：患者扫描医院提供的二维码进行注册，注册过程中，患者需提供其病历号，系统根据病历号自动获取患者基本信息。

（2）登录：已注册的患者可以登录系统进行线上抓药，未册的患者系统拒绝其登陆。

（3）确认处方：患者登录后，可以查看医生开具的所有处方。患者选择需要抓药的处方和数量（需要抓几副药），同时说明是否需要煎制。选择取药方式：自行到店取药或者送药上门,若选择送药上门，患者需要提供提供收货人姓名、联系方式和收货地址。系统自动计算本次抓药的费用，患者可以使用微信或支付宝等支付方式支付费用。支付成功之后，处方被发送给药师进行药品配制。

（4）处理处方：药师根据处方配置好药品。若患者要求煎制，药师对配置好的药品进行煎制。煎制完成，药师将该处方设置为已完成。若患者选择的是自行取药，取药后确认已取药。

（5）药品派送：处方完成后，对于选择送药上门的患者，系统将给快递人员发送药品配送信息，等待快递人员取药；并给患者发送收获验证码。

（6）送药上门：快递人员将配制好的药品送到患者指定的收获地址。患者收货时，向快递人员出示收获验证码，快递人员使用该验证码确认药品已送到。

现采用面向对象分析与设计方法开发上述系统，得到如图 3-1 所示的用例图以及图 3-2 所示的类图。

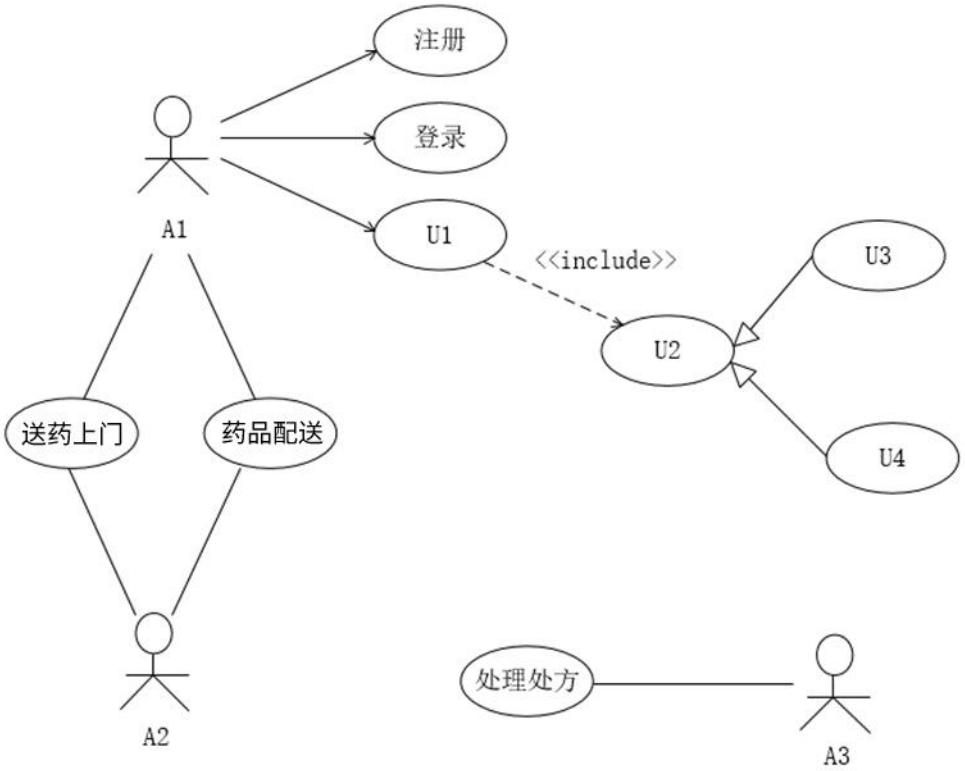


图 3-1 用例图

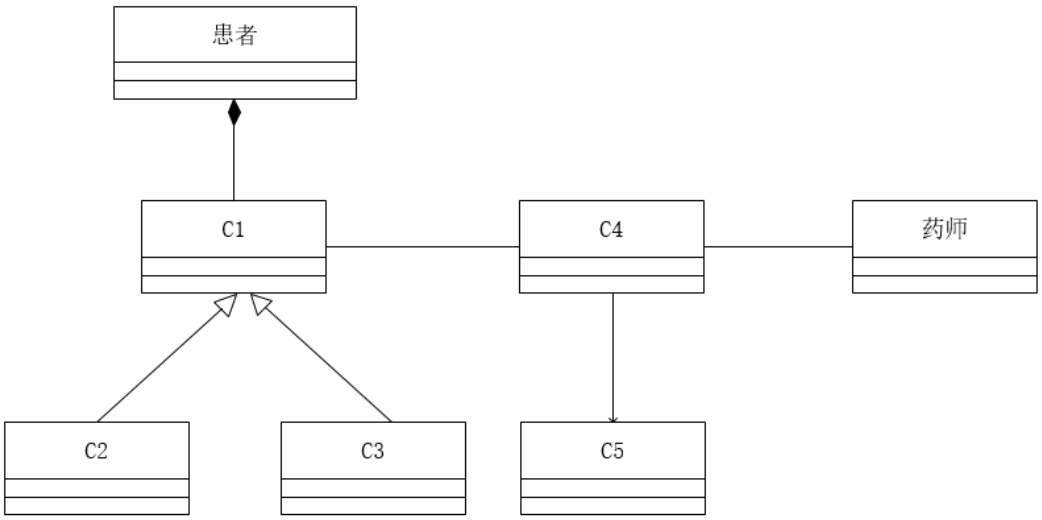


图 3-2 类图

【问题 1】（7 分）

根据说明中的描述，给出图 3-1 中 A1～A3 所对应的参与者名称和 U1～U4 处所对应的用例名称。

【问题 2】（5 分）

根据说明中的描述，给出图 3-2 中 C1～C5 所对应的类名。

【问题 3】（3 分）

简要解释用例之间的 include、extend 和 generalize 关系的内涵。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 2，将解答填入答题纸的对应栏内。

【说明】

凸多边形是指多边形的任意两点的连线均落在多边形的边界或内部。相邻的点连线落在多边形边界上，称为边；不相邻的点连线落在多边形内部，称为弦。假设任意两点连线上均有权重，凸多边形最优三角剖分问题定义为：求将凸多边形划分为不相交的三角形集合，且各三角形权重之和最小的剖分方案。每个三角形的权重为三条边权重之和。

假设 N 个点的凸多边形点编号为 V_1，V_2，..... ，V_N,若在 V_k 处将原凸多边形划分为一个三角形 $V_1V_kV_N$，两个子多边形 V_1，V_2，... ，V_k 和 V_k，V_{k+1}，... ，V_N 得到一个最优的剖分方案，则该最优剖分方案应该包含这两个子凸边形的最优剖分方案。用 m[i][j]表示 V_{i-1}，V_i，... ,V_j 构成的凸多边形的最优剖分方案的权重，S[i][j]记录剖分该凸多边形的 k 值。

则：

$$m[i][j]=\begin{cases}0, i \geq j \\ \min\limits_{i\leq k<j}\{m[i][k]+m[k+1][j]+W(V_{i-1}V_kV_j)\}, i<j\end{cases}$$

其中：$W(V_{i-1}V_kV_j) = W_{i-1,k} + W_{k,j} + W_{j,i-1}$ 为三角形 $V_{i-1}V_kV_j$ 的权重，$W_{i-1,k}$，$W_{k,j}$，$W_{j,i-1}$ 分别为该三角形三条边的权重。求解凸多边形的最优剖分方案，即求解最小剖分的权重及对应的三角形集。

【C 代码】

```
#include<stdio.h>
#define N 6 // 凸多边形规模
int m[N + 1][N + 1]; // m[i][j]表示多边形 Vi-1 到 Vj 最优三角剖分的权值
int S[N + 1][N + 1]; // S[i][j]记录多边形 Vi-1 到 Vj 最优三角剖分的 k 值
int W[N + 1][N + 1]; // 凸多边形的权重矩阵，在 main 函数中输入

/* 三角形的权重 a, b, c, 三角形的顶点下标 */
int get_ triangle_weight(int a, int b, int c) {
    return W[a][b] + W[b][c] + W[c][a];
}
```

```
/* 求解最优值 */
void triangle_partition() {
    int i, r, k, j;
    int temp;
    /* 初始化 */
for (i = 1; i <= N; i ++ ) {
        m[i][i] = 0;
    }
    /* 自底向上计算 m, S */
    for (r = 2; __(1)__; r ++ ) { /* r 为子问题规模 */
        for (i = 1; k <= N – r + 1; i ++ ) {
            __(2)__;
            m[i][j] = m[i][j] + m[i + 1][j] +
                      get_triangle_weight(i - 1, i, j); /* k = j */
            S[i][j] = i;
            for (k = j + 1; k < j; k ++ ) { /*计算 m[i][j]的最小代价 */
                temp = m[i][k] + m[k + 1][j] +
                       ge_triangle_ weight(i - 1, k, j);
                if (__(3)__) { /* 判断是否最小值 */
                    m[i][j] = temp;
                    S[i][j] = k;
                }
            }
        }
    }
}
```

```
/* 输出剖分的三角形 i, j：凸多边形的起始点下标 */
void print_triangle(int i, nt j) {
    if (i == j) return;
    print_triangle(i, S[i][j]);
    print_triangle(  (4)  );
    printf("V%d--V%d--V%d\n", i - 1, S[i][j], j);
}
```

【问题 1】（8 分）

根据题干说明，填充 C 代码中的空（1）～（4）。

【问题 2】（7 分）

根据题干说明和 C 代码，该算法采用的设计策略为（5）。

算法的时间复杂度为（6），空间复杂度为（7）（用 O 表示）

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入__（n）__处的字句写在答题纸的对应栏内。

【说明】

层叠菜单是窗口风格的软件系统中经常采用的一种系统功能组织方式。层叠菜单（如图 6-1 示例）中包含的可能是一个菜单项（直接对应某个功能），也可能是一个子菜单，现在采用组合（composite）设计模式实现层叠菜单，得到如图 6-2 所示的类图

层叠菜单（如图 6-1 示例）暂缺

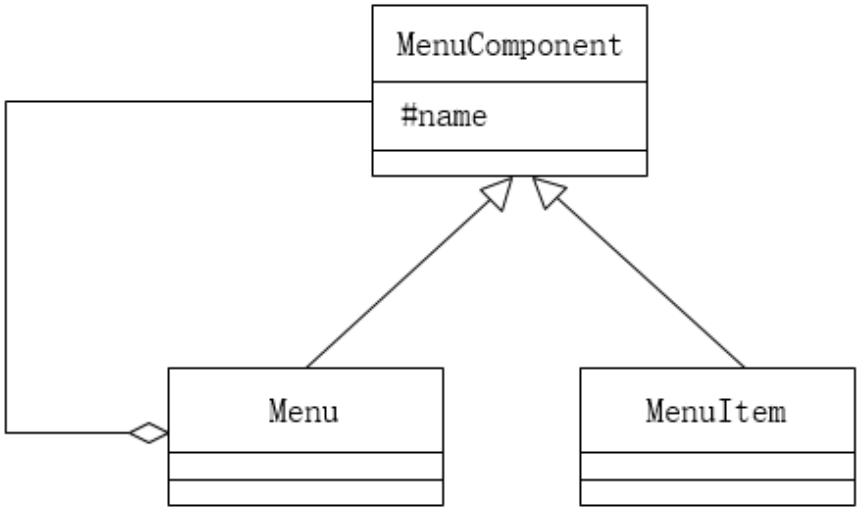


图 6-2 类图

【Java 代码】

```
import java.util.*;

abstract class MenuComponent {  // 构成层叠菜单的元素
    __(1)__ String name;            // 菜单项或子菜单名称
    public void printName() { System.out.println(name); }
    public __(2)__;
    public abstract boolean removeMenuElement(MenuComponent element);
    public __(3)__;
}
```

```
class MenuItem extends MenuComponent {
    public MenuItem(String name) { this.name=name; }
    public boolean addMenuElement(MenuComponent element) { return false; }
    public boolean removeMenuElement(MenuComponent element) {
        return false;
    }
    public List<MenuComponent> getElement(){ return null; }
}

class Menu extends MemuComponent {
    private  (4)  ;
    public Menu(String name) {
        this.name = name;
        this.elementList = new ArrayList<MenuComponent>;
    }
    public boolean addMenuElement(MenuComponent element) {
        return elementList.add(element);
    }
    public boolean removeMenuElement(MenuComponent element) {
        return elementList.remove(element);
    }
    public List<MenuComponent> getElement() { return elementList; }
}
```

```
class CompositeTest {
    public static void main(String[] args) {
        MenuComponent mainMenu = new Menu("Insert");
        MenuComponent subMenu = new Menu("Chart");
        MenuComponent element = new MenuItem("On This Sheet");
        __(5)__;
        subMenu.addMenuElement(element);
        printMenus(mainMenu);
    }

    private static void printMenus(MenuComponent ifile) {
        ifile.printName();
        List<MenuComponent> children = ifile.getElement();
        if (children == null) return;
        for(MenuComponent element; children) {
            printMenus(element);
        }
    }
}
```

全国计算机技术与软件专业技术资格（水平）考试

2021 年下半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1．在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。

2．在答题纸的指定位置填写准考证号、出生年月日和姓名。

3．答题纸上除填写上述内容外只能写解答。

4．本试卷共 6 道题，试题一至试题四是必答题，试题五至试题六选答 1 道。每题 15 分，满分 75 分。

5．解答时字迹务必清楚，字迹不清时，将不评分。

6．仿照下面的例题，将解答写在答题纸的对应栏内。

例题

2021 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是（1）月（2）日。

因为正确的解答是“11 月 6 日”，故在答题纸的对应栏内写上“11”和“6”（参看下表）。

例题	解答栏
（1）	11
（2）	6

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某现代农业种植基地为进一步提升农作物种植过程的智能化，欲开发智慧农业平台，集管理和销售于一体，该平台的主要功能有:

（1）信息维护。农业专家对农作物、环境等监测数据的监控处理规则进行维护。

（2）数据采集。获取传感器上传的农作物长势、土壤墒情、气候等连续监测数据，解析后将监测信息进行数据处理、可视化和存储等操作。

（3）数据处理。对实时监测信息根据监控处理规则进行监测分析，将分析结果进行可视化并进行存储、远程控制，对历史监测信息进行综合统计和预测，将预测信息进行可视化和存储。

（4）远程控制。根据监控处理规则对分析结果进行判定，依据判定结果自动对控制器进行远程控制。平台也可以根据农业人员提供的控制信息对控制器进行远程控制。

（5）可视化。实时向农业人员展示监测信息：实时给农业专家展示统计分析结果和预测信息或根据农业专家请求进行展示。

现采用结构化方法对智慧农业平台进行分析与设计，获得如图 1-1 所示的上下文数据流图和图 1-2 所示的 0 层数据流图。

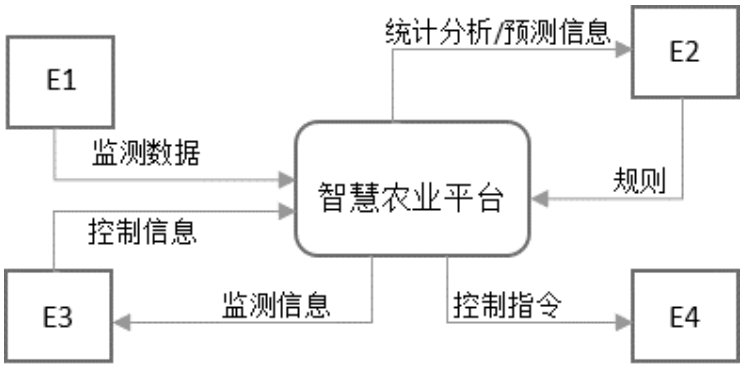


图 1-1 上下文数据流图

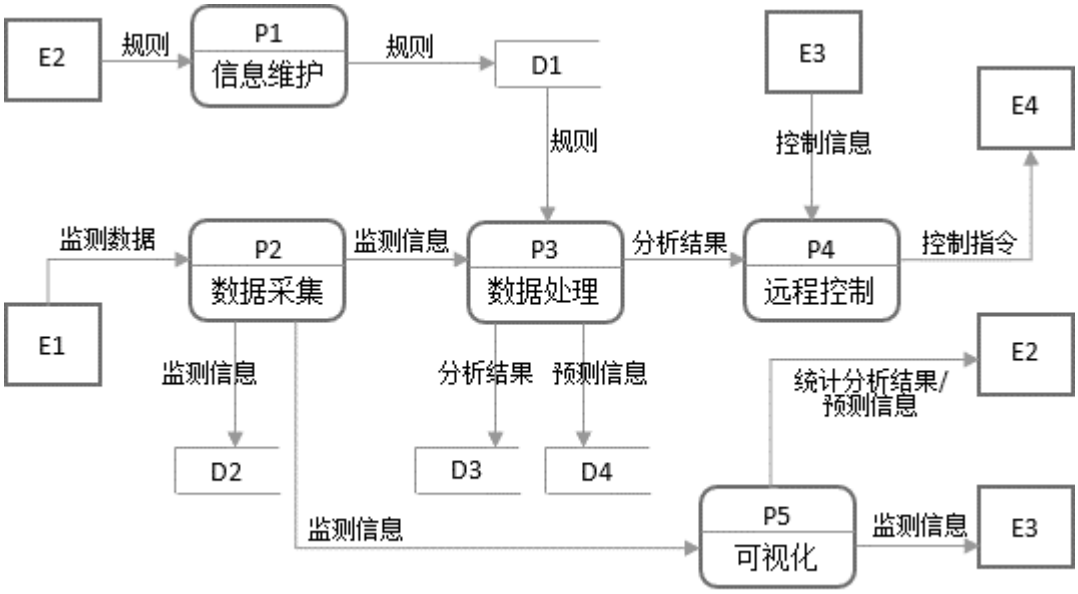


图 1-2　0 层数据流图

【问题 1】（4 分）

使用说明中的词语，给出图 1-1 中的实体 E1～E4 的名称。

【问题 2】（4 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1～D4 的名称。

【问题 3】（4 分）

根据说明和图中术语，补充图 1-2 中缺失的数据流及其起点和终点。

【问题 4】（3 分）

根据说明，“数据处理”可以分解为哪些子加工？进一步进行分解时，需要注意哪三种常见的错误？

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某汽车维修公司为了便于管理车辆的维修情况，拟开发一套汽车维修管理系统，请根据下述需求描述完成该系统的数据库设计。

【需求分析结果】

（1）客户信息包括：客户号、客户名、客户性质、折扣率、联系人、联系电话。客户性质有个人或单位。客户号唯一标识客户关系中的每一个元组。

（2）车辆信息包括：车牌号、车型、颜色和车辆类别。一个客户至少有一辆车，一辆车只属于一个客户。

（3）员工信息包括：员工号、员工名、岗位、电话、家庭住址。其中，员工号唯一标识员工关系中的每一个元组。岗位有业务员、维修工、主管。业务员根据车辆的故障情况填写维修单。

（4）部门信息包括：部门号、名称、主管和电话。其中，部门号唯一确定部门关系的每一个元组。每个部门只有一名主管，但每个部门有多名员工，每名员工只属于一个部门。

（5）维修单信息包括：维修单号、车牌号、维修内容、工时。其中，维修单号唯一标识维修单关系中的每一个元组。一个维修工可以接多张维修单，但一张维修单只对应一个维修工。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

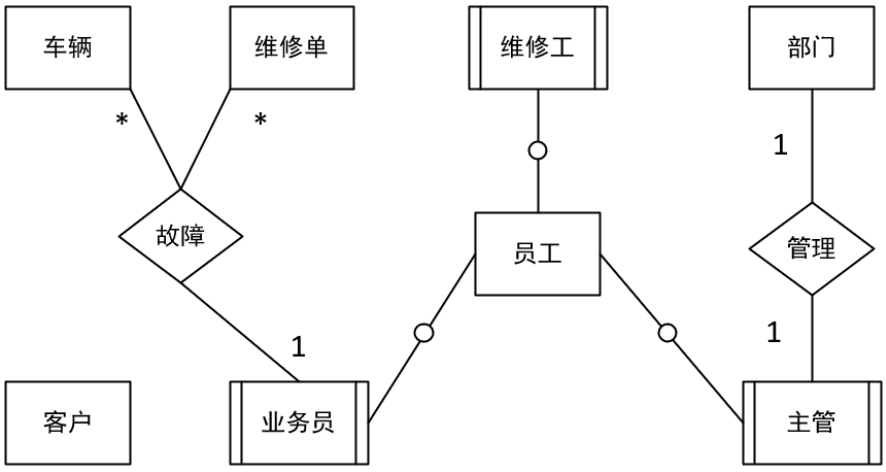


图2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

客户（客户号，客户名，___（a）___，折扣率，联系人，联系电话）

车辆（车牌号，___（b）___，车型，颜色，车辆类别）

员工（员工号，员工名，岗位，___（c）___，电话，家庭住址）

部门（部门号，名称，主管，电话）

维修单（维修单号，___（d）___，维修内容，工时）

【问题 1】（6 分）

根据问题描述，补充 3 个联系，完善图 2-1 的实体联系图。联系名可以用联系 1、联系 2 和联系 3 代替，联系的类型为 1:1、1:n 和 m:n（或 1:1、1:*和*:*）

【问题 2】（4 分）

根据题意，将关系模式中的空（a）～（d）的属性补充完整，并填入答题纸对应的位置上。

【问题 3】（2 分）

分别给出车辆关系和维修单关系的主键和外键。

【问题 4】（3 分）

如果一张维修单涉及多项维修内容，需要多个维修工来处理，那么哪个联系类型会发生何种变化？你认为应该如何解决这一问题？

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某游戏公司欲开发一款吃金币游戏。游戏的背景为一种回廊式迷宫（Maze），在迷宫的不同位置上设置有墙。迷宫中有两种类型的机器人（Robos）：小精灵（PacMan）和幽灵（Ghost）。游戏的目的就是控制小精灵在迷宫中游走，吞吃迷宫路径上的金币，且不能被幽灵抓到。幽灵在迷宫中游走，并会吃掉遇到的小精灵。机器人游走时，以单位距离的倍数计算游走路径的长度。当迷宫中至少存在一个小精灵和一个幽灵时，游戏开始。

机器人上有两种传感器，使机器人具有一定的感知能力。这两种传感器分别是：

（1）前向传感器（FrontSensor）。探测在机器人当前位置的左边、右边和前方是否有墙（机器人遇到墙时，必须改变游走方向）。机器人根据前向传感器的探测结果，决定朝哪个方向运动。

（2）近距离传感器（ProxiSensor）。探测在机器人的视线范围内（正前方）是否存在隐藏的金币或幽灵。近距离传感器并不报告探测到的对象是否正在移动以及朝哪个方向移动。但是如果近距离传感器的连续两次探测结果表明被探测对象处于不同的位置，则可以推到出该对象在移动。

另外，每个机器人都设置有一个计时器（Timer），用于支持执行预先定义好的定时事件。机器人的动作包括：原地向左或向右旋转 90°、向前或向后移动。

建立迷宫：用户可以使用编辑器（Editor）编写迷宫文件，建立用户自定义的迷宫。将迷宫文件导入游戏系统建立用户自定义的迷宫。

现采用面向对象分析与设计方法开发该游戏，得到如图 3-1 所示的用例图以及图 3-2 所示的初始类图。



图3-1 用例图

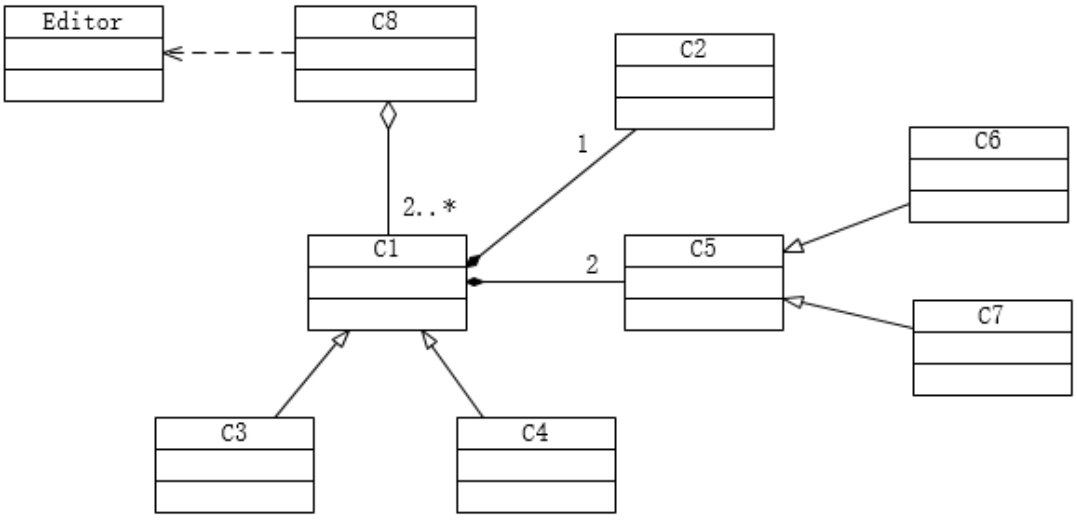


图3-2 类图

【问题 1】（3 分）

根据说明中的描述，给出图 3-1 中 U1～U3 所对应的用例名。

【问题 2】（4 分）

图 3-1 中用例 U1～U3 分别与哪个（哪些）用例之间有关系，是何种关系？

【问题 3】（8 分）

根据说明中的描述，给出图 3-2 中 C1～C8 所对应的类名。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 2，将解答填入答题纸的对应栏内。

【说明】

生物学上通常采用编辑距离来定义两个物种 DNA 序列的相似性，从而刻画物种之间的进化关系。具体来说，编辑距离是指将一个字符串变换为另一个字符所需要的最小操作次数。操作有三种，分别为：插入一个字符、删除一个字符以及将一个字符修改为另一个字符。用字符数组 str1 和 str2 分别表示长度为 len1 和 len2 的字符串，定义二维数组 d 记录求解编辑距离的子问题最优解，则该二维数组可以递归定义为：

$$d[i][j]=\begin{cases} i & 若len2=0 \\ j & 若len1=0 \\ d[i-1][j-1] & 若str[i-1]=str2[j-1] \\ \min\{d[i-1][j]+1, d[i][j-1]+1, d[i-1][j-1]+1\} & 若str[i-1]\neq str2[j-1] \end{cases}$$

【C 代码】

C 代码

下面是算法的 C 语言实现。

（1）常量和变量说明

A，B：两个字符数组

d[][]：二维数组

i，j：循环变量

temp：临时变量

（2）C 程序

```
#include <stdio.h>
#define N 100

char A[N] = "CTGA";
char B[N] = "ACGCTA";
int d[N][N];
```

```
int min(int a, int b) {
    return a < b ? a : b;
}

int editdistance(char *str1, int len1, char *str2, int len2) {
    int i, j;
    int diff;
    int temp;
    for (i = 0; i <= len1; i ++ ) {
        d[i][0] = i;
    }
    for (j = 0; j <= len2; j ++ ) {
        __(1)__;
    }
    for (i = 1; i <= len1; i ++ ) {
        for (j = 1; j <= len2; j ++ ) {
            if ( __(2)__ ) {
                d[i][j] = d[i - 1][j - 1];
            } else {
                temp = min(d[i - 1][j] + 1, d[i][j - 1] + 1);
                d[i][j] = min(temp, __(3)__);
            }
        }
    }
    return __(4)__;
}
```

【问题 1】（8 分）

根据说明和 C 代码，填充 C 代码中的空（1）～（4）。

【问题 2】（4 分）

根据说明和 C 代码，算法采用了（5）设计策略，时间复杂度为（6）（用 O 符合表示，两个字符串的长度分别用 m 和 n 表示）。

【问题 3】（3 分）

已知两个字符串 A=“CTGA”和 B=“ACGCTA”，根据说明和 C 代码，可得出这两个字符串的编辑距离为（7）。

试题五（共 15 分）

阅读下列说明和 C++代码，将应填入__（n）__处的字句写在答题纸的对应栏内。

【说明】

享元（FlyWeight）模式主要用于减少创建对象的数量，以降低内存占用，提高性能。先要开发一个网络围棋程序，允许多个玩家联机下棋。由于只有一台服务器，为节省内存空间，采用享元模式实现该程序，得到如图 5-1 所示的类图。

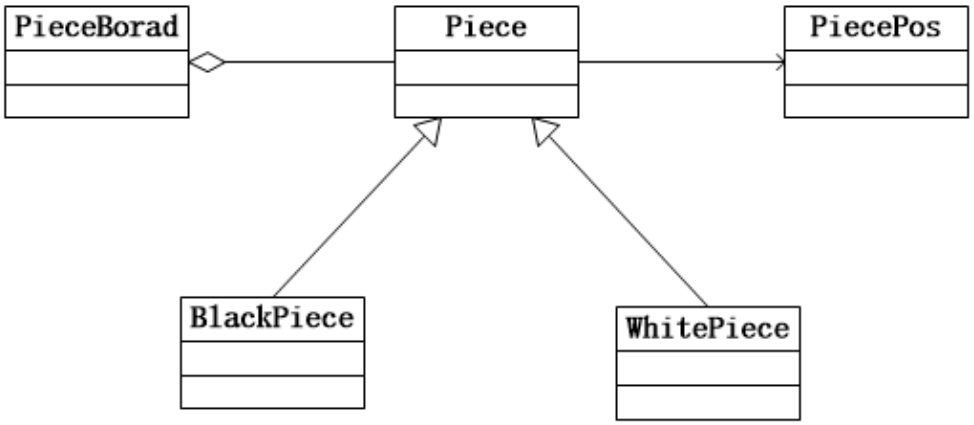


图5-1　类图

【C++代码】

```
#include <iostream>
#include <vector>
using namespace std;

enum PieceColor {BLACK, WHITE}; // 棋子颜色
```

```
class PiecePos { // 棋子位置
private :
    int x;
    int y;
public :
    PiecePos(int a, int b); x(a), y(b) {}
    int getX() { return x; }
    int getY() { return y; }
};

class Piece { // 棋子定义
protected :
    PieceColor m_color; // 颜色
    PiecePos m_pos; // 位置
public :
    Piece(PieceColor color, PiecePos pos) : m_color(color), m_pos(pos) {}
    __(1)__;
};

class BlackPiece : public Piece {
public :
    BlackPiece(PieceColor color, PiecePos pos) : Piece(color, pos) {}
    void Draw() { cout << "draw a black piece" << endl; }
};

class WhitePiece : public Piece {
public :
    WhitePiece(PieceColor color, PiecePos pos) : Piece(color, pos) {}
    void Draw() { cout << "draw a white piece" << endl;
};
```

```
class PieceBoard {
private :
    vector<  (2)  > m_vecPiece; // 棋盘上已有的棋子
    string m_blackName; // 黑方名称
    string m_whiteName; // 白方名称
public :
    PieceBoard(string black, string white) :
    m_blackName(black), m_whiteName(white) {}
    // 一步棋，在棋盘上放一颗棋子
    void SetPiece(PieceColor color, PiecePos pos) {
        (3) piece = NULL;
        if (color == BLACK) { // 放黑子
            piece = new BlackPiece(color, pos); // 获取一颗黑子
            cout << m_blackName << "在位置（" << pos.getX() << ","
            << pos.getY() << "）";
            (4) ;
        } else { // 放白子
            Piece = new WhitePiece(color, pos); // 获取一颗白子
            cout << m_whiteName << "在位置（" << pos.getX() << ","
            << pos.getY() << "）";
            (5) ;
        }
        m_vecPiece.push_back(piece);
    }
};
```

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入（n）处的字句写在答题纸的对应栏内。

【说明】

享元（FlyWeight）模式主要用于减少创建对象的数量，以降低内存占用，提高性能。先要开发一个网络围棋程序，允许多个玩家联机下棋。由于只有一台服务器，为节省内存空间，采用享元模式实现该程序，得到如图 6-1 所示的类图。

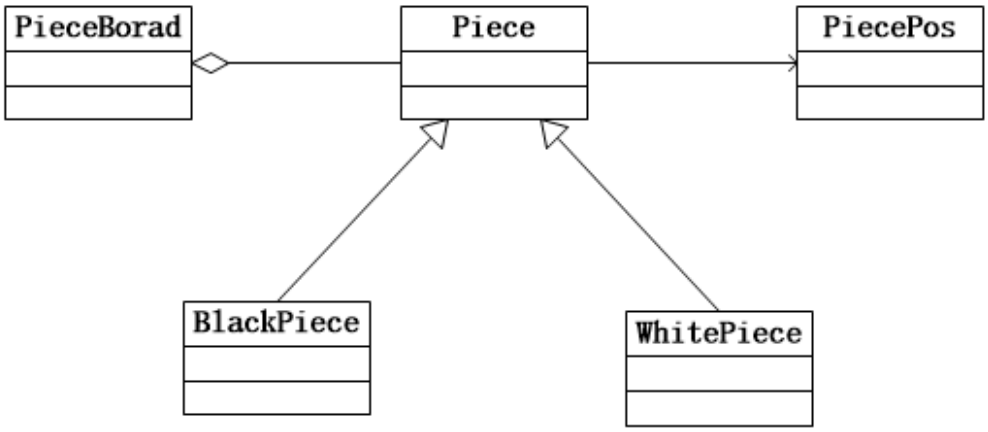


图6-1 类图

【Java 代码】

```
import java.util.*;

enum PieceColor { BLACK, WHITE } // 棋子颜色

class PiecePos { // 棋子位置
    private int x;
    private int y;
    public PiecePos(int a, int b) { x = a; y = b; }
    public int getX() { return x; }
    public int getY() { return y; }
}
```

```
abstract class Piece { // 棋子定义
    protected PieceColor m_color; // 颜色
    protected PiecePos m_pos; // 位置
    public Piece(PieceColor color, PiecePos pos) {
        m_color = color;
        m_pos = pos;
    }
    __(1)__;
}

class BlackPiece extends Piece {
    public BlackPiece(PieceColor color, PiecePos pos) {
        super(color, pos);
    }
    public void draw() { System.out.println("draw a blackpiece"); }
}

class WhitePiece extends Piece {
    public WhitePiece(PieceColor color, PiecePos pos) {
        super(color, pos);
    }
    public void draw() { System.out.println("white a blackpiece"); }
}
```

```
class PieceBoard { // 棋盘上已有的棋子
    private static final ArrayList<(2)> m_arrayPiece = new ArrayList
    private String m_blackName; // 黑方名称
    private String m_whiteName; // 白方名称
    public PieceBoard(String black, String white) {
        m_blackName = black;
        m_whiteName = white;
    }

    // 一步棋，在棋盘上放一颗棋子
    public void SetPiece(PieceColor color, PiecePos pos) {
        __(3)__ piece = null;
        if (color == PieceColor.BLACK) { // 放黑子
            piece = new BlackPiece(color, pos); // 获取一颗黑子
            System.out.println(m_blackName + "在位置(" + pos.getX() + ","
                               + pos.getY() + ")");
            __(4)__;
        } else { // 放白子
            piece = new WhitePiece(color, pos); // 获取一颗白子
            System.out.println(m_whiteName + "在位置(" + pos.getX() + ","
                               + pos.getY() + ")");
            __(5)__;
        }
        m_arrayPiece.add(piece);
    }
}
```

全国计算机技术与软件专业技术资格（水平）考试

2022 年上半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1．在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。

2．在答题纸的指定位置填写准考证号、出生年月日和姓名。

3．答题纸上除填写上述内容外只能写解答。

4．本试卷共 6 道题，试题一至试题四是必答题，试题五至试题六选答 1 道。每题 15 分，满分 75 分。

5．解答时字迹务必清楚，字迹不清时，将不评分。

6．仿照下面的例题，将解答写在答题纸的对应栏内。

例题

2022 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是（1）月（2）日。

因为正确的解答是“5 月 28 日”，故在答题纸的对应栏内写上“5”和“28”（参看下表）。

例题	解答栏
（1）	5
（2）	28

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

某公司欲开发一款外卖订餐系统，集多家外卖平台和商户为一体，为用户提供在线浏览餐品、订餐和配送等服务。该系统的主要功能是：

（1）入驻管理。用户注册；商户申请入驻，设置按时间段接单数量阈值等。系统存储商户/用户信息。

（2）餐品管理。商户对餐品的基本信息和优惠信息进行发布、修改、删除。系统存储相关信息。

（3）订餐。用户浏览商户餐单，选择餐品及数量后提交订餐请求。系统存储订餐订单。

（4）订单处理。收到订餐请求后，向外卖平台请求配送。外卖平台接到请求后发布配送单，由平台骑手接单，外卖平台根据是否有骑手接单返回接单状态。若外卖平台接单成功，系统给支付系统发送支付请求，接收支付状态。支付成功，更新订单状态为已接单，向商户发送订餐请求并由商户打印订单，给用户发送订单状态；若支付失败，更新订单状态为下单失败，向外卖平台请求取消配送，向用户发送下单失败。若系统接到外卖平台返回接单失败或超时未返回接单状态，则更新订单状态为下单失败，向用户发送下单失败。

（5）配送。商户备餐后，由骑手取餐配送给用户。送达后由用户扫描骑手出示的订单上的配送码后确认送达，订单状态更改为已送达，并发送给商户。

（6）订单评价。用户可以对订单餐品、骑手配送服务进行评价，推送给对应的商户、所在外卖平台，商户和外卖平台对用户的评价进行回复。系统存储评价。

现采用结构化方法对外卖订餐系统进行分析与设计，获得如图 1-1 所示的上下文数据流图和图 1-2 所示的 0 层数据流图。

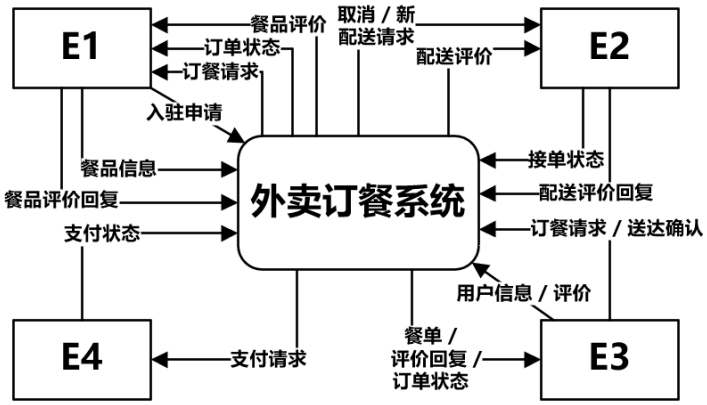


图 1-1 上下文数据流图

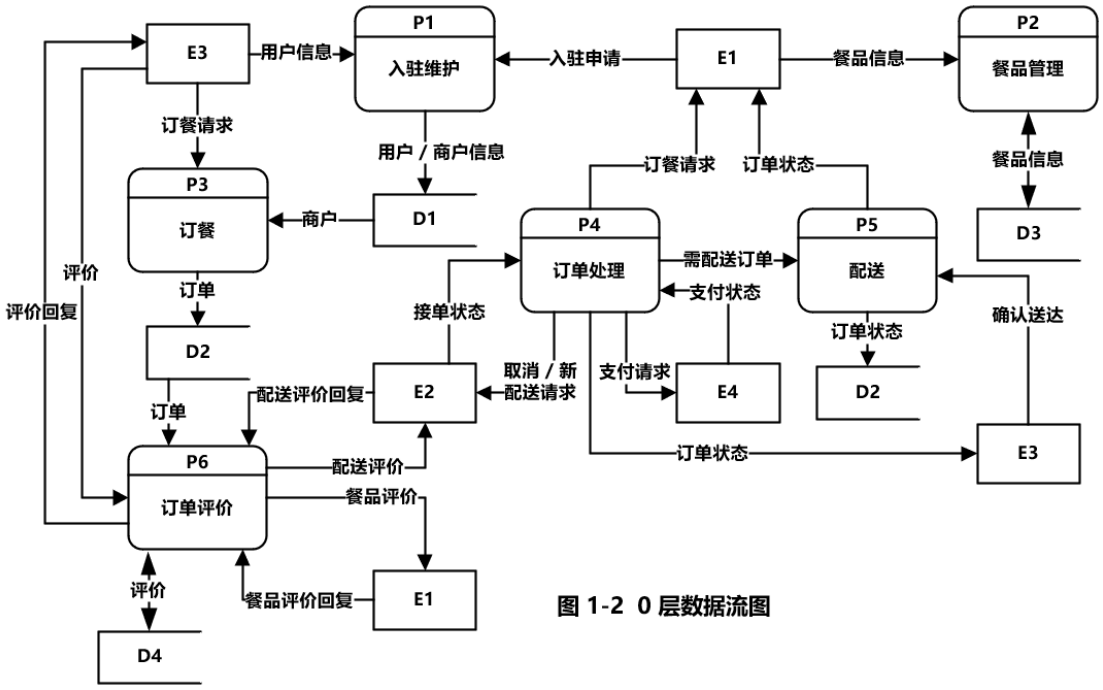


图 1-2 0 层数据流图

【问题 1】（4 分）

使用说明中的词语，给出图 1-1 中的实体 E1～E4 的名称。

【问题 2】（4 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1～D4 的名称。

【问题 3】（4 分）

根据说明和图中术语，补充图 1-2 中缺失的数据流及其起点和终点。

【问题 4】（3 分）

根据说明，采用结构化语言对“订单处理”的加工逻辑进行描述。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

为了提高接种工作，提高效率，并为了抗击疫情提供疫苗接种数据支撑，需要开发一个信息系统，下述需求完成该系统的数据库设计。

【需求分析结果】

（1）记录疫苗供应商的信息，包括供应商名称，地址和一个电话。

（2）记录接种医院的信息，包括医院名称、地址和一个电话。

（3）记录接种者个人信息，包括姓名、身份证号和一个电话。

（4）记录接种者疫苗接种信息，包括接种医院信息，被接种者信息，疫苗供应商名称和接种日期，为了提高免疫力，接种者可能需要进行多次疫苗接种，（每天最多接种一次，每次都可以在全市任意一家医院进行疫苗接种）。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

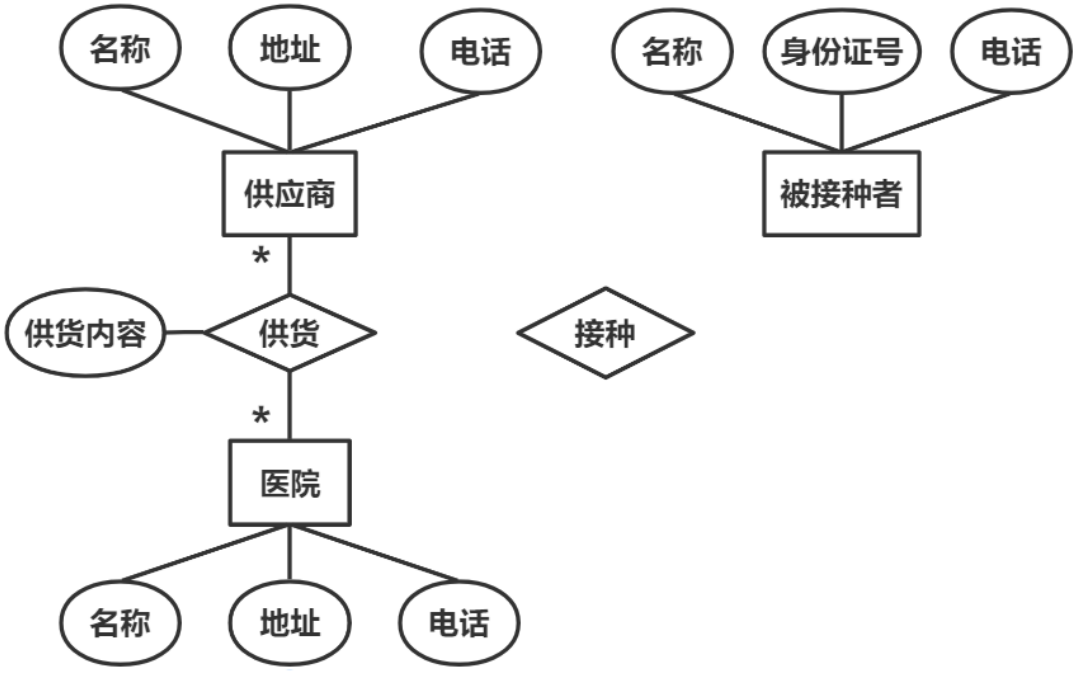


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

供应商（供应商名称，地址，电话）

医院（医院名称，地址，电话）

供货（供货商名称，　（a）　，供货内容）

被接种者（姓名，身份证号，电话）

接种（接种者身份证号，　（b）　，医院名称、供应商名称）

【问题 1】（4 分）

根据问题描述，补充图 2-1 的实体联系图（不增加新的实体）。

【问题 2】（4 分）

根据题意，补充逻辑结构设计结果中的（a），（b）两处空缺，并标注主键和外键完整性约束。

【问题 3】（7 分）

若医院还兼有核酸检测的业务，检测时可能需要进行多次核酸检测（每天最多检测一次），但每次都可以在全市任意一家医院进行检测。

请在图 2-1 中增加“被检测者”实体及相应的属性。医院与被检测者之间的“检测”联系及必要的属性，并给出新增加的关系模式。

“被检测者”实体包括姓名、身份证号、地址和一个电话。

“检测”联系包括检测日期和检测结果等。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某公司的人事部门拥有一个地址簿（AddressBook）管理系统（AddressBookSystem），用于管理公司所有员工的地址记录（PersonAddress）。员工的地址记录包括：姓名、住址、城市、省份、邮政编码以及联系电话等信息。

管理员可以完成对地址簿中地址记录的管理操作，包括：

（1）管理地址记录。根据公司的人员变动情况，对地址记录进行添加、修改、删除等操作。

（2）排序。按照员工姓氏的字典顺序或邮政编码对地址簿中的所有记录进行排序。

（3）打印地址记录。以邮件标签的格式打印一个地址单独的地址簿。

系统会对地址记录进行管理，为便于管理，管理员在系统中为公司的不同部门建立员工的地址簿的操作，包括：

（1）创建地址簿。新建一个地址簿并保存。

（2）打开地址簿。打开一个已有的地址簿。

（3）修改地址簿。对打开的地址簿进行修改并保存。

系统将提供一个 GUI（图形用户界面）实现对地址簿的各种操作。

现采用面向对象方法分析并设计该地址簿管理系统，得到如图 3-1 所示的用例图以及图 3-2 所示的类图。

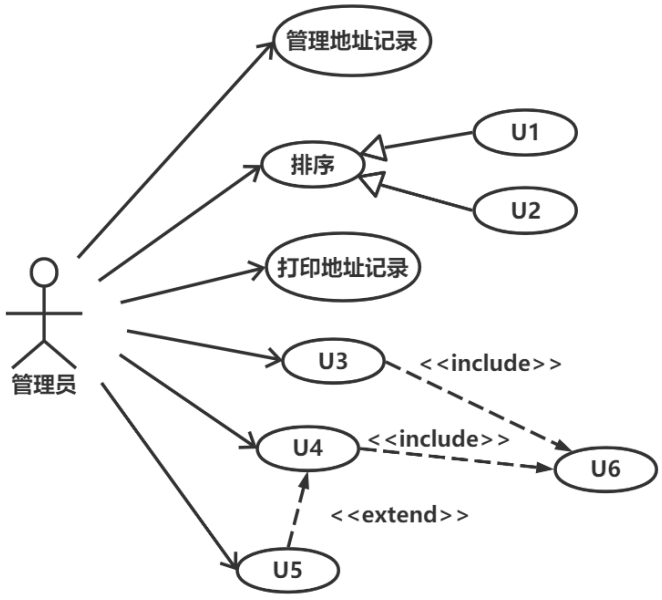


图 3-1 用例图

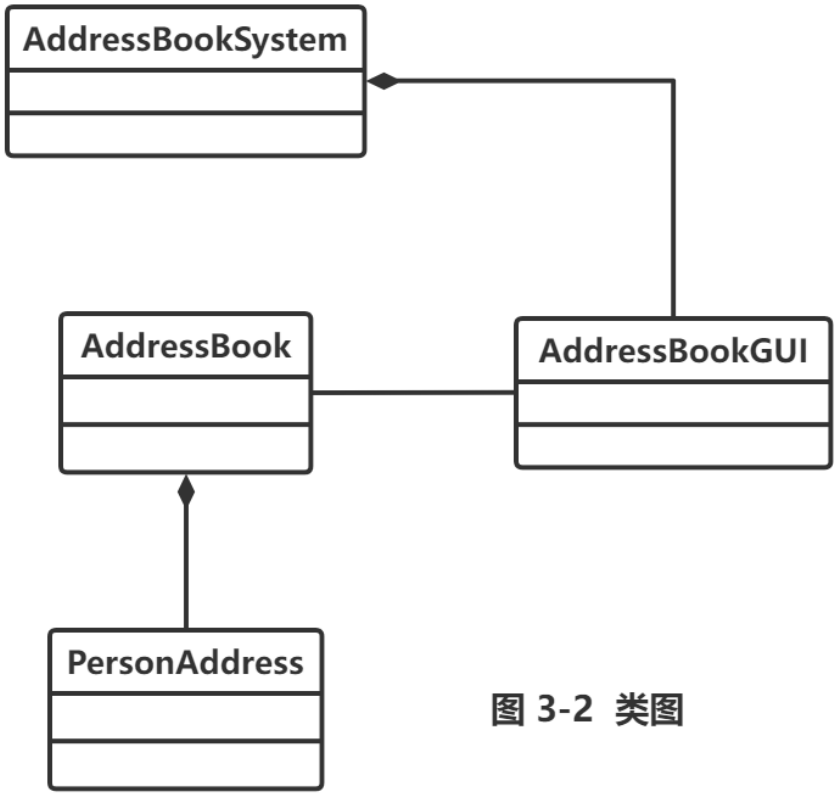


图 3-2 类图

【问题 1】（6 分）

根据说明中的描述，给出图 3-1 中 U1～U6 所对应的用例名。

【问题 2】（5 分）

根据说明中的描述，给出图 3-2 中类 AddressBook 的主要属性和方法以及类 PersonAddress 的主要属性（可以使用说明中的文字）

【问题 3】（4 分）

根据说明中的描述以及图 3-1 所示的用例图，请简要说明 include 和 extend 关系的含义是什么？

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某工程计算中要完成多个矩阵相乘（链乘）的计算任务，对矩阵相乘进行以下说明。

（1）两个矩阵相乘要求第一个矩阵的列数等于第二个矩阵的行数，计算量主要由进行乘法运算的次数决定。假设采用标准的矩阵相乘算法，计算$A_{n*m} * B_{n*p}$，需要$m * n * p$次乘法运算，即时间复杂度为$O(m * n * p)$

（2）矩阵相乘满足结合律，多个矩阵相乘时不同的计算顺序会产生不同的计算量。以矩阵$A1_{5*100}$，$A2_{100*8}$，$A3_{8*50}$三个矩阵相乘为例，若按$(A1 * A2) * A3$计算，则需要进行$5 * 100 * 8 + 5 * 8 * 50 = 6000$次乘法运算，若按$A1 * (A2 * A3)$计算，则需要进行$100 * 8 * 50 + 5 * 100 * 50 = 65000$次乘法运算。

矩阵连乘问题可描述为：给定n个矩阵，对较大的n，可能计算的顺序数量非常庞大，用蛮力法确定计算顺序是不实际的。经过对问题进行分析，发现矩阵连乘问题具有最优子结构，即若$A1 * A2 * \cdots * An$的一个最优计算顺序从第k个矩阵处断开，即分为$A1 * A2 * \cdots * Ak$和$Ak + 1 * Ak + 2 * \cdots * An$两个子问题，则该最优解应该包含$A1 * A2 * \cdots * Ak$的一个最优计算顺序和$Ak + 1 * Ak + 2 * \cdots * An$的一个最优计算顺序。据此构造递归式：

$$\text{cost}[i][\text{j}] = \begin{cases} 0 & \text{if i=j} \\ \min_{\text{i}\leqslant\text{k}<\text{j}} \text{cost[i][k]} + \text{cost[k+1][j]} + \text{p}_{\text{i}} * \text{p}_{\text{k+1}} * \text{p}_{\text{j+1}} & \text{if i<j} \end{cases}$$

其中，`cost[i][j]`表示$Ai + 1 * Ai + 2 * \cdots * Aj + 1$的最优计算的代价。最终需要求解 `cost[0][n - 1]`

【C 代码】

算法实现采用自底向上的计算过程。首先计算两个矩阵相乘的计算量，然后依次计算 3 个矩阵、4 个矩阵、…、n个矩阵相乘的最小计算量及最优计算顺序。下面是算法的 C 语言实现。

（1）主要变量说明

`n`：矩阵数、

`seq[]`：矩阵维数序列

`cost[][]`：二维数组，长度为$n * n$，其中元素 `cost[i][j]`表示$Ai + 1 * Ai + 2 * \cdots * Aj + 1$的最优计算的计算代价

trace[][]：二维数组，长度为$n*n$，其中元素 trace[i][j]表示$Ai+1*Ai+2*\cdots*Aj+1$的最优计算对应的划分位置，即k

（2）函数 cmm

```
#define N 100
int cost[N][N];
int trace[N][N];
int cmm(int n, int seq[]) {
    int tempCost;
    int tempTrace;
    int i, j, k, p;
    int temp;
    for (i = 0; i < n; i ++ ) { cost[i][i] = 0; }
        for (p = 1; p < n; p ++ ) {
            for (i = 0; i < n - p; i ++ ) {
                (1) ;
                tempCost = -1;
                for (k = i; (2) ; k ++ ) {
                    temp = (3) ;
                    if(tempCost == -1 || tempCost > temp) {
                        tempCost = temp;
                        tempTrace = k;
                    }
                }
                cost[i][j] = tempCost;
                (4) ;
            }
        }
    return cost[0][n - 1];
}
```

【问题 1】（8 分）

根据以上说明和 C 代码，填充 C 代码中的空（1）～（4）。

【问题 2】（4 分）

根据以上说明和 C 代码，该问题采用了__（5）__算法设计策略，时间复杂度为__（6）__。（用 O 符号表示）

【问题 3】（3 分）

考虑实例 n = 4，各个矩阵的维数：A1 为 15 * 5，A2 为 5 * 10，A3 为 10 * 20，A4 为 20 * 25，即维数序列为 15，5，10，20，25。则根据上述 C 代码得到的一个最优计算顺序为__（7）__（用加括号方式表示计算顺序），所需要的乘法运算次数为__（8）__。

试题五（共 15 分）

阅读下列说明和 C++代码，将应填入__（n）__处的字句写在答题纸的对应栏内。

【说明】

在软件系统中，通常都会给用户提供取消、不确定或者错误的操作，允许将系统回复到原先的状态。现使用备忘录（Memento）模式实现该要求，得到如图 5-1 所示的类图。

Memento 包含了要被恢复的状态。Originator 创建并在 Memento 中存储状态。CareTaker 负责从 Memento 中恢复状态。

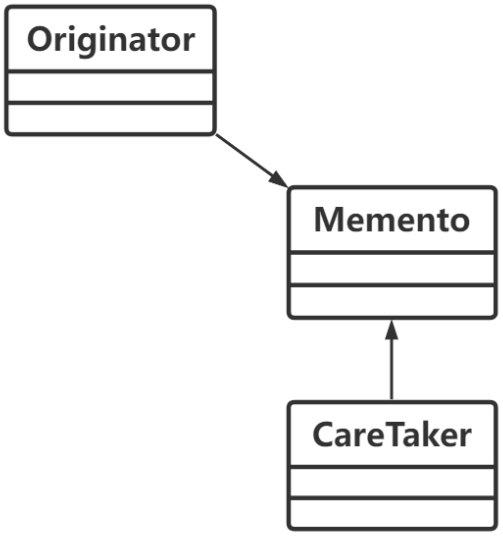


图 5-1 类图

【C++代码】

```
#include <iostream>
#include <string>
#include <vector>

using namespace std;
```

```
class Memento {
    private:
        string state;

    public:
        Memento(string state) { this->state = state; }
        string getState() { return state; }
};

class Originator {
    private:
        string state;

    public:
        void setState(string state) { this->state = state; }
        string getState() { return state; }
        Memento saveStateToMemento() { return __(1)__; }
        void getStateFromMemento(Memento Memento) { state = __(2)__; }
};

class CareTaker {
    private:
        vector<Memento> mementoList;
    public:
        void __(3)__ { mementoList.push_back(state); }
        __(4)__ { return mementoList[index]; }
};
```

```
int main() {
    Originator *originator = new Originator();
    CareTaker *careTaker = new CareTaker();
    originator->setState("State #1");
    originator->setState("State #2");
    careTaker->add(__(5)__);
    originator->setState("State #3");
    careTaker->add(__(6)__);
    originator->setState("State #4");

    cout << "Current State: " + originator->getState() << endl;
    originator->getStateFromMemento(careTaker->get(0));
    cout << "First saved State: " + originator->getState() << endl;
    originator->getStateFromMemento(careTaker->get(1));
    cout << "Second saved State: " + originator->getState() << endl;
}
```

试题六（共 15 分）

阅读下列说明和 Java 代码，将应填入__（n）__处的字句写在答题纸的对应栏内。

【说明】

在软件系统中，通常都会给用户提供取消、不确定或者错误的操作，允许将系统回复到原先的状态。现使用备忘录（Memento）模式实现该要求，得到如图 6-1 所示的类图。

Memento 包含了要被恢复的状态。Originator 创建并在 Memento 中存储状态。CareTaker 负责从 Memento 中恢复状态。

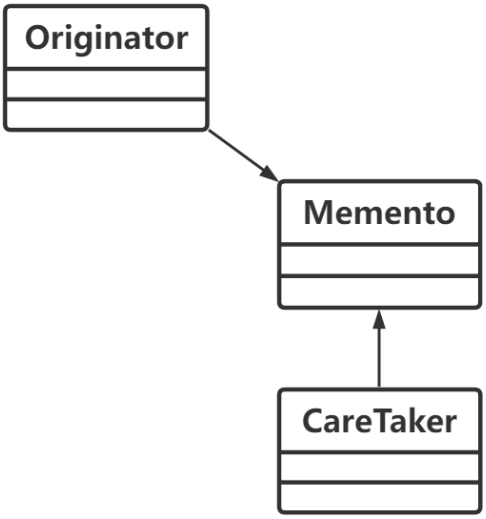


图 6-1 类图

【Java 代码】

```
import java.util.*;


class Memento {
    private String state;
    public Memento(String state) {
        this.state = state;
    }

    public String getState() {
        return state;
    }
}
```

```
class Originator {
    private String state;
    public void setState(String state) {
        this.state = state;
    }

    public String getState() {
        return state;
    }

    public Memento saveStateToMemento() {
        return __(1)__;
    }

    public void getStateFromMemento(Memento Memento) {
        state = __(2)__;
    }
}

class CareTaker {
    private List<Memento> mementoList = new ArrayList<Memento>();

    public __(3)__ {
        mementoList.add(state);
    }

    public __(4)__ {
        return mementoList.get(index);
    }
}
```

```
class MementoPaneDemos {
    public static void main(String[] args) {
        Originator originator = new Originator();
        CareTaker careTaker = new CareTaker();
        originator.setState("State #1");
        originator.setState("State #2");
        careTaker.add(  (5)  );
        originator.setState("State #3");
        careTaker.add(  (6)  );
        originator.setState("State #4");

        System.out.println("Current State: " + originator.getState());
        originator.getStateFromMemento(careTaker.get(0));
        System.out.println("First saved State: " + originator.getState());
        originator.getStateFromMemento(careTaker.get(1));
        System.out.println("Second saved State: " + originator.getState());
    }
}
```

全国计算机技术与软件专业技术资格（水平）考试

2022 年下半年 软件设计师 下午试卷

（考试时间 14:00～16:30 共 150 分钟）

请按下述要求正确填写答题卡

1．在答题纸的指定位置填写你所在的省、自治区、直辖市、计划单列市的名称。

2．在答题纸的指定位置填写准考证号、出生年月日和姓名。

3．答题纸上除填写上述内容外只能写解答。

4．本试卷共 6 道题，试题一至试题四是必答题，试题五至试题六选答 1 道。每题 15 分，满分 75 分。

5．解答时字迹务必清楚，字迹不清时，将不评分。

6．仿照下面的例题，将解答写在答题纸的对应栏内。

例题

2022 年下半年全国计算机技术与软件专业技术资格（水平）考试日期是（1）月（2）日。

因为正确的解答是“11 月 5 日”，故在答题纸的对应栏内写上“11”和“5”（参看下表）。

例题	解答栏
（1）	11
（2）	5

试题一（共 15 分）

阅读下列说明和数据流图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

随着新能源车数量的迅猛增长，全国各地电动汽车配套充电桩急速增长，同时也带来了充电桩计量准确性的问题。充电桩都需要配备相应的电能计量和电费计费功能，需要对充电计量准确性强制进行检定。现需开发计量检定云端软件，其主要功能是：

（1）数据接收。接收计量装置上报的充电数据，即充电过程中电压、电流、电能等充电监测数据和计量数据（充电监测数据为充电桩监测的数据，计量数据为计量装置计量的数据，以秒为间隔单位），接收计量装置心跳数据，并分别进行存储。

（2）基础数据维护。管理员对充电桩、计量检定装置等基础数据进行维护。

（3）数据分析。实现电压、电流、电能数据的对比，进行误差分析，记录充电桩的充电误差，供计量装置检定。系统根据计量检测人员给出的查询和统计条件展示查询统计结果。

（4）充电桩检定。分析充电误差：计量检测人员根据误差分析结果和检定信息记录，对充电桩进行检定，提交检定结果：系统更新充电桩中的检定信息（检定结果和检定时间），并存储于检定记录。

（5）异常告警。检测计量装置心跳，当心跳停止时，向管理员发出告警。

（6）检定信息获取。供其它与充电桩相关的第三方服务查询充电桩中的检定信息。

现采用结构化方法对计量检定云端软件进行分析与设计，获得如图 1-1 所示的上下文数据流图和图 1-2 所示的 0 层数据流图。

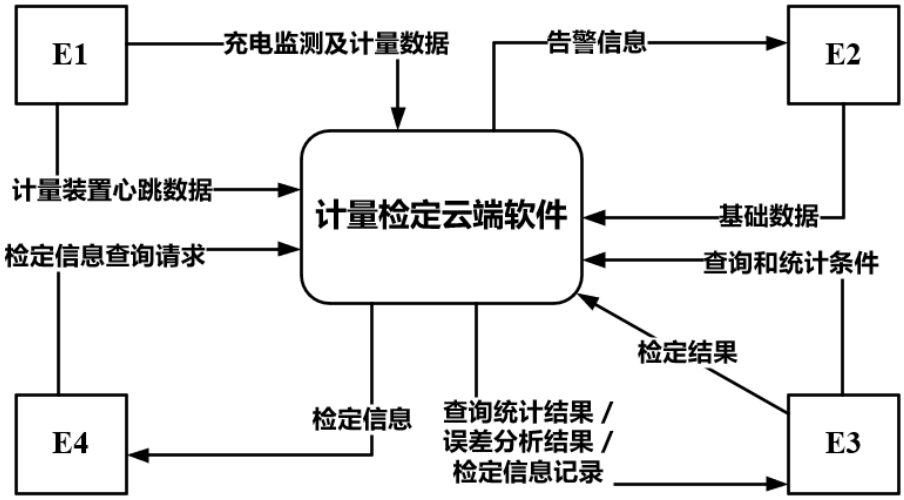


图 1-1 上下文数据流图

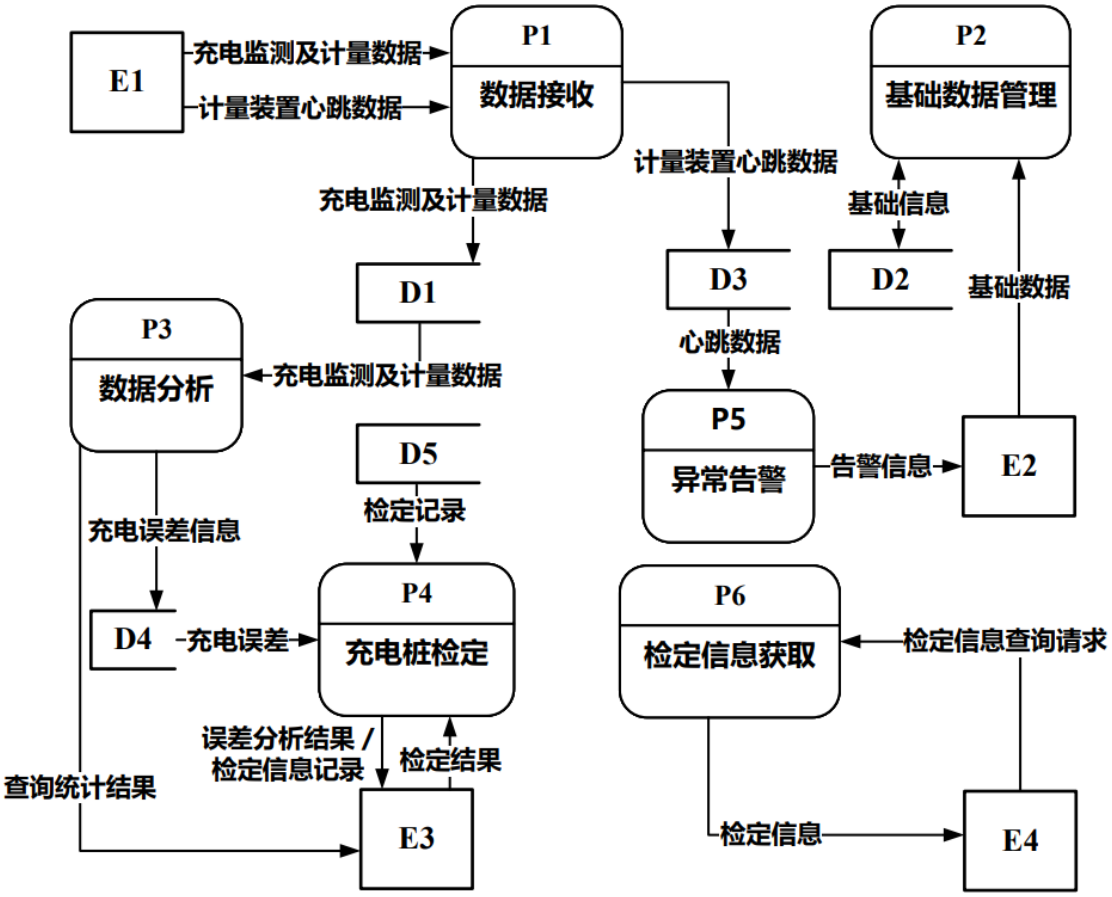


图 1-2 0 层数据流图

【问题 1】（4 分）

使用说明中的词语，给出图 1-1 中的实体 E1～E4 的名称。

【问题 2】（5 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1～D5 的名称。

【问题 3】（4 分）

根据说明和图中术语，补充图 1-2 中缺失的数据流及其起点和终点。

【问题 4】（2 分）

根据说明，给出“充电监测与计量数据”数据流的组成。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某营销公司为了便于对各地的分公司及专卖店进行管理，拟开发一套业务管理系统，请根据下述需求描述完成该系统的数据库设计。

【需求分析结果】

（1）分公司信息包括：分公司编号、分公司名、地址和电话。其中，分公司编号唯一确定分公司关系的每一个元组。每个分公司拥有多家专卖店，每家专卖店只属于一个分公司。

（2）专卖店信息包括：专卖店号、专卖店名、店长、分公司编号、地址、电话，其中店号唯一确定专卖店关系中的每一个元组。每家专卖店只有一名店长，负责专卖店的各项业务；每名店长只负责一家专卖店：每家专卖店有多名职员，每名职员只属于一家专卖店。

（3）职员信息包括：职员号、职员名、专卖店号、岗位、电话、薪资。其中，职员号唯一标识职员关系中的每一个元组。岗位有店长、营业员等。

【概念模型设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

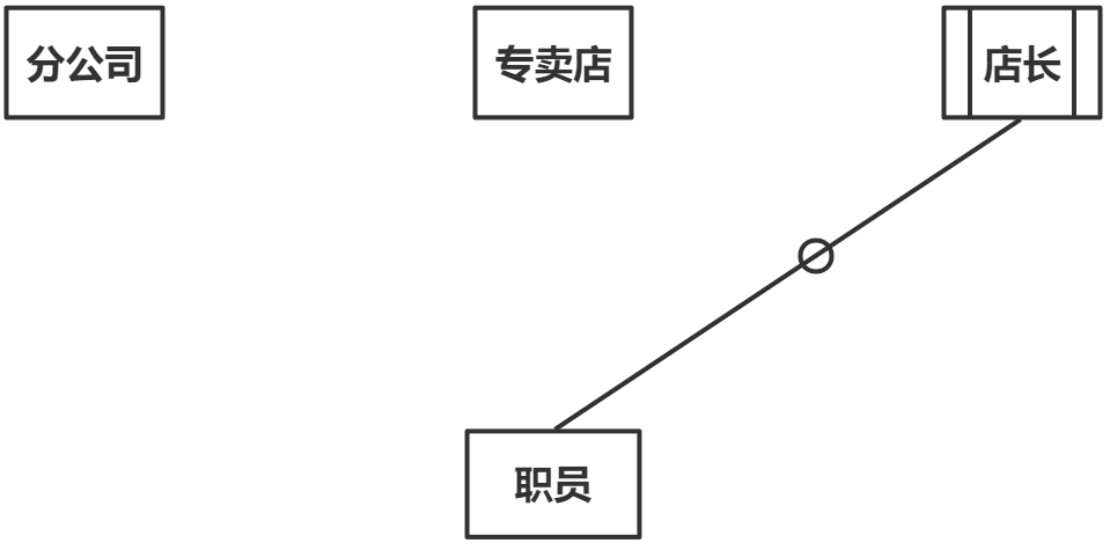


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

分公司（分公司编号，分公司名，地址，电话）

专卖店（专卖店号，专卖店名，___（a）___，地址，电话）

职员（职员号，职员名，___（b）___，岗位，电话，薪资）

【问题 1】（6 分）

根据需求描述，图 2-1 实体联系图中缺少三个联系。请在答题纸对应的实体联系图中补充三个联系及联系类型。

注：联系名可用联系 1、联系 2、联系 3；也可根据你对题意的理解取联系名。

【问题 2】（6 分）

（1）将关系模式中的空___（a）___、___（b）___的属性补充完整，并填入答题纸对应的位置上。

（2）专卖店关系的主键：___（c）___和外键：___（d）___。

职员关系的主键：___（e）___和外键：___（f）___。

【问题 3】（3 分）

（1）为了在紧急情况发生时，能及时联系到职员的家人，专卖店要求每位职员至少要填写一位紧急联系人的姓名、与本人关系和联系电话。根据这种情况，在图 2-1 中还需添加的实体是___（g）___，职员关系与该实体的联系类型为___（h）___。

（2）给出该实体的关系模式。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

图 3-1 所示为某软件系统中一个温度控制模块的界面。界面上提供了两种温度计量单位，即华氏度（Farechet）和摄氏度（Celsius）。软件支持两种计量单位之间的自动换算，即若输入一个华氏度的温度，其对应的摄氏度温度值会自动出现在摄氏度的显示框内，反之亦然。

用户可以通过该界面上的按钮 Raise（升高温度）和 Lower（降低温度）来改变温度的值。界面右侧是个温度计，将数字形式的温度转换成温度计上的制度比例进行显示。当温度值改变时，温度计的显示也随之同步变化。

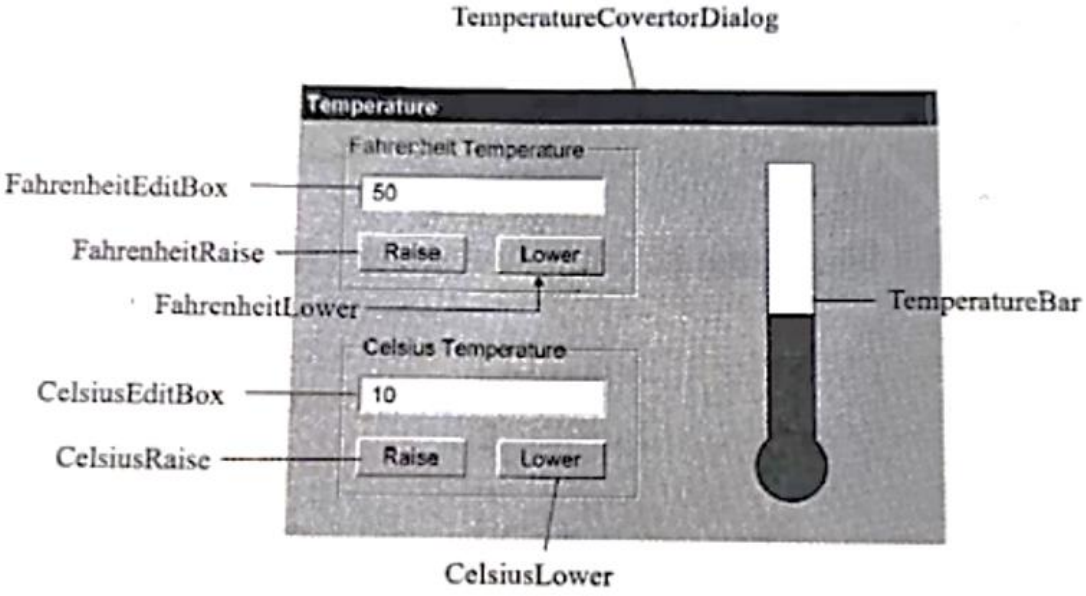


图 3-1 温度控制模块的界面

现采用面向对象方法现实该温度控制模板，得到如图 3-2 所示的用例图和图 3-3 所示的类图。

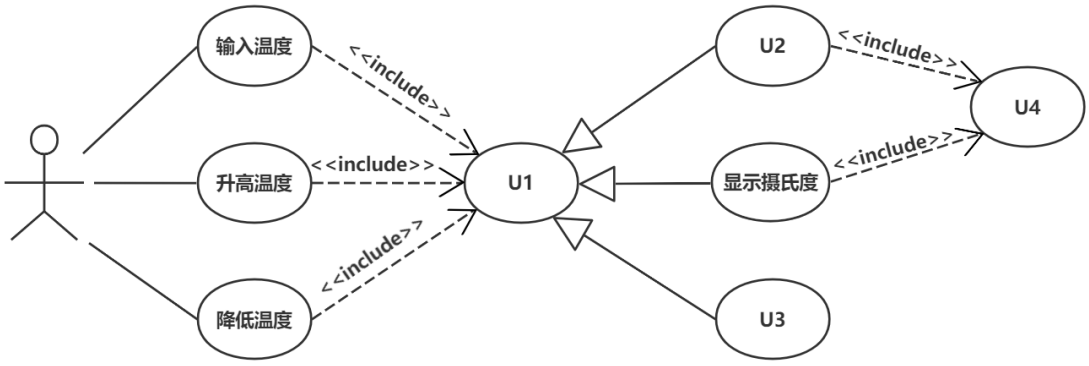


图 3-2 用例图

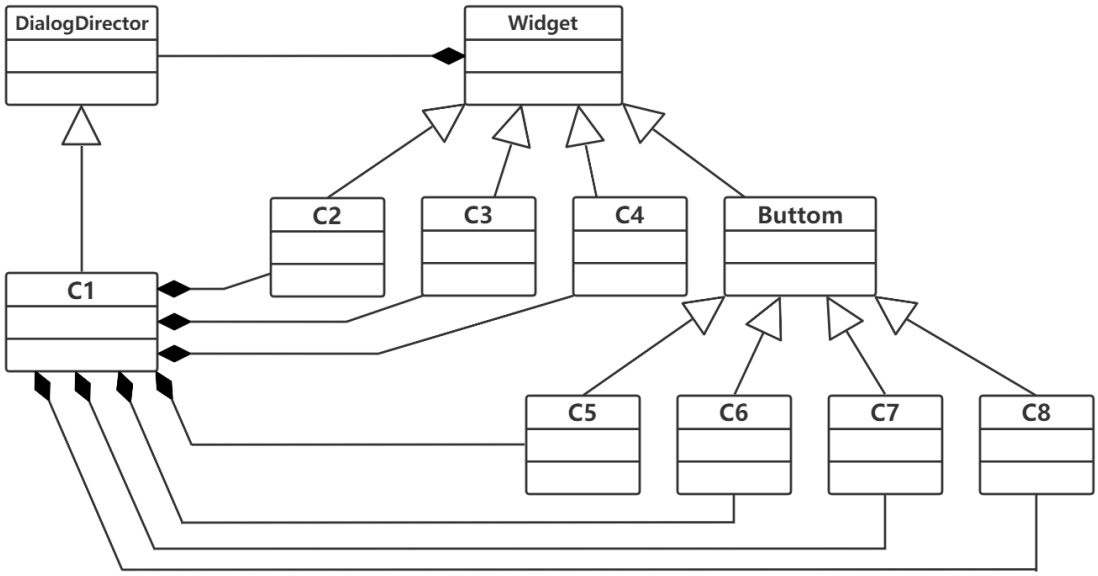


图 3-3 类图

【问题 1】（6 分）

根据说明中的描述，给出图 3-2 中 U1～U4 所对应的用例名。

【问题 2】（5 分）

根据说明中的描述，给出图 3-3 中 C1～C8 所对应的类名（类名使用图 3-1 中标注的词汇）。

【问题 3】（4 分）

现需将图 3-1 所示的界面改造为一个更为通用的 GUI 应用，能够实现任意计量单位之间的换算，例如千克和克之间的换、厘米和英寸之间的换算等等。为了实现这个新的需求，可以在图 3-3 所示的类图上增加哪种设计模式？请解释选择该设计模式的原因（不超过 50 字）。

试题四（共 15 分）

阅读下列说明和 C 代码，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

排序是将一组无序的数据元素调整为非递减顺序的数据序列的过程，堆排序是一种常用的排序算法。用顺序存储结构存储堆中元素。非递减堆排序的步骤是：

（1）将含 n 个元素的待排序数列构造成一个初始大顶堆，存储在数组 R（R[1]，R[2]，...，R[n]）中。此时堆的规模为 n，堆顶元素 R[1]就是序列中最大的元素，R[n]是堆中最后一个元素。

（2）将堆顶元素和堆中最后一个元素交换，最后一个元素脱离堆结构，堆的规模减 1，将堆中剩余的元素调整成大顶堆;

（3）重复步骤（2），直到只剩下最后一个元素在堆结构中，此时数组 R 是一个非递减的数据序列。

【C 代码】

下面是该算法的 C 语言实现。

（1）主要变量说明

n：待排序的数组长度

R[]：待排序数组，n 个数放在 R[1]，R[2]，...，R[n]中

（2）C 程序

```
#include <stdio.h>
#define MAXITEM 100
```

```
/*
 * 调正堆
 * R：待排序数组；
 * V：结点编号，以 v 为根的二叉树，R[v] ≥ R[2v]，R[v] ≥ R[2v + 1]，
 * 且其左子树和右子树都是大顶堆；
 * n：堆结构的规模，即堆中的元素数
 */
void Heapify(int R[MAXITEM], int v, int n) {
    int i, j;
    i = v;
    j = 2 * i;

    R[0] = R[i];
    while (j <= n) {
        if (j < n && R[j] < R[j + 1]) {
            j ++ ;
        }
        if ( __（1）__ ) {
            R[i] = R[j];
            i = j;
            j = 2 * i;
        } else {
            j = n + 1;
        }
    }
    R[i] = R[0];
}
```

```
/* 堆排序，R 为待排序数组：n 为数组大小 */
void HeapSort(int R[MAXITEM], int n) {
    int i;

    for (i = n / 2; i >= 1; i -- ) {
        （2）;
    }

    for (i = n; （3）; i -- ) {
        R[0] = R[i];
        R[i] = R[1];
        （4）;
        Heapify(R, 1, i - 1);
    }
}
```

【问题 1】（8 分）

根据以上说明和 C 代码，填充 C 代码中的空（1）~（4）。

【问题 2】（2 分）

根据以上说明和 C 代码，算法的时间复杂度为（5）（用 O 符号表示）。

【问题 3】（5 分）

考虑数据序列 R=（7，10，13，15，4，20，19，8），n=8，则构建的初始大顶堆为（6），

第一个元素脱离堆结构，对剩余元素再调整成大顶堆后的数组 R 为（7）。

试题五（共 15 分）

阅读下列说明和 Java 代码，将应填入（n）处的字句写在答题纸的对应栏内。

【说明】

Facade（外观）模式是一种通过为多个复杂子系统提供一个一致的接口，而使这些子系统更加容易被访问的模式，以医院为例，就医时患者需要与医院不同的职能部门交互，完成挂号、门诊、取药等操作。为简化就医流程，设置了一个接待员的职位，代患者完成上述就医步骤，患者则只需与接待员交互即可。如图 5-1 给出了以外观模式实现该场景的类图。

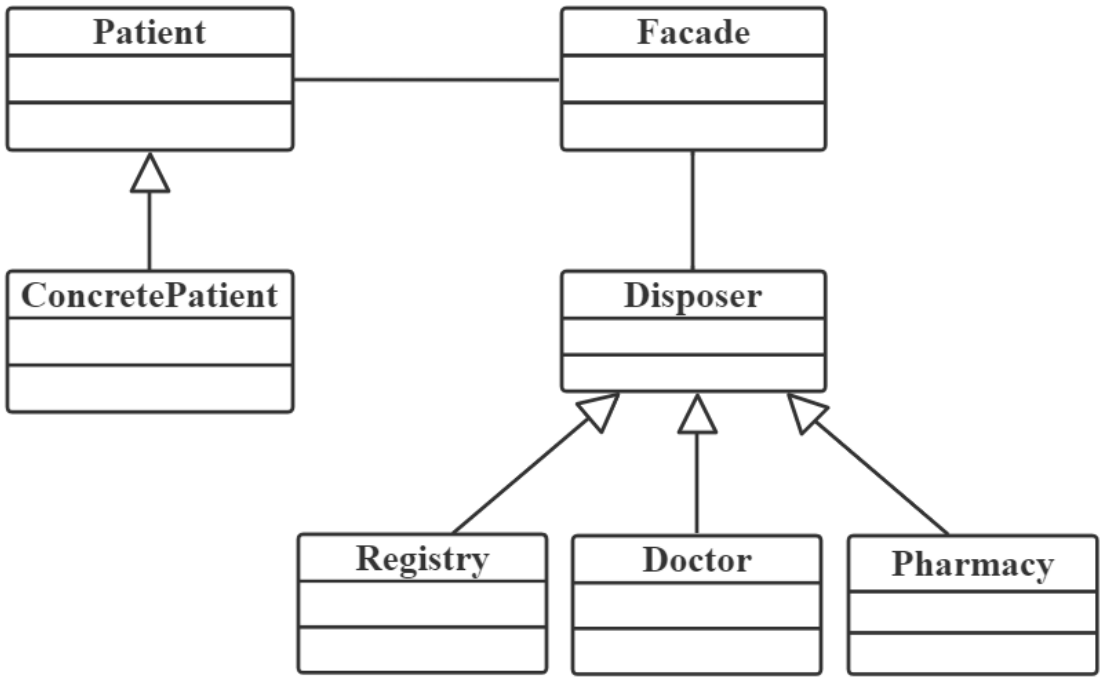


图 5-1 类图

【Java 代码】

```
import java.util.*;


interface Patient {
    （1）;
}
interface Disposer {
    （2）;
}
```

```
class Registry implements Disposer { // 挂号
    public void dispose(Patient patient) {
        System.out.println("I am registering..." + patient.getName());
    }
}

class Doctor implements Disposer { // 医生门诊
    public void dispose(Patient patient) {
        System.out.println("I am diagnosing..." + patient.getName());
    }
}

class Pharmacy implements Disposer { // 取药
    public void dispose(Patient patient) {
        System.out.println("I am giving medicine... " + patient.getName());
    }
}
```

```
class Facade {
    private Patient patient;

    public Facade(Patient patient) {
        this.patient = patient;
    }

    void dispose() {
        Registry registry = new Registry();
        Doctor doctor = new Doctor();
        Pharmacy pharmacy = new Pharmacy();

        registry.dispose(patient);
        doctor.dispose(patient);
        pharmacy.dispose(patient);
    }
}

class ConcretePatient implements Patient {
    private String name;

    public ConcretePatient(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

```
public class FacadeTest {
    public static void main(String[] args) {
        Patient patient = （3）;
        （4） f = （5）;
        （6）;
    }
}
```

试题六（共 15 分）

阅读下列说明和 C++代码，将应填入__（n）__处的字句写在答题纸的对应栏内。

【说明】

Facade（外观）模式是一种通过为多个复杂子系统提供一个一致的接口，而使这些子系统更加容易被访问的模式，以医院为例，就医时患者需要与医院不同的职能部门交互，完成挂号、门诊、取药等操作。为简化就医流程，设置了一个接待员的职位，代患者完成上述就医步骤，患者则只需与接待员交互即可。如图 6-1 给出了以外观模式实现该场景的类图。

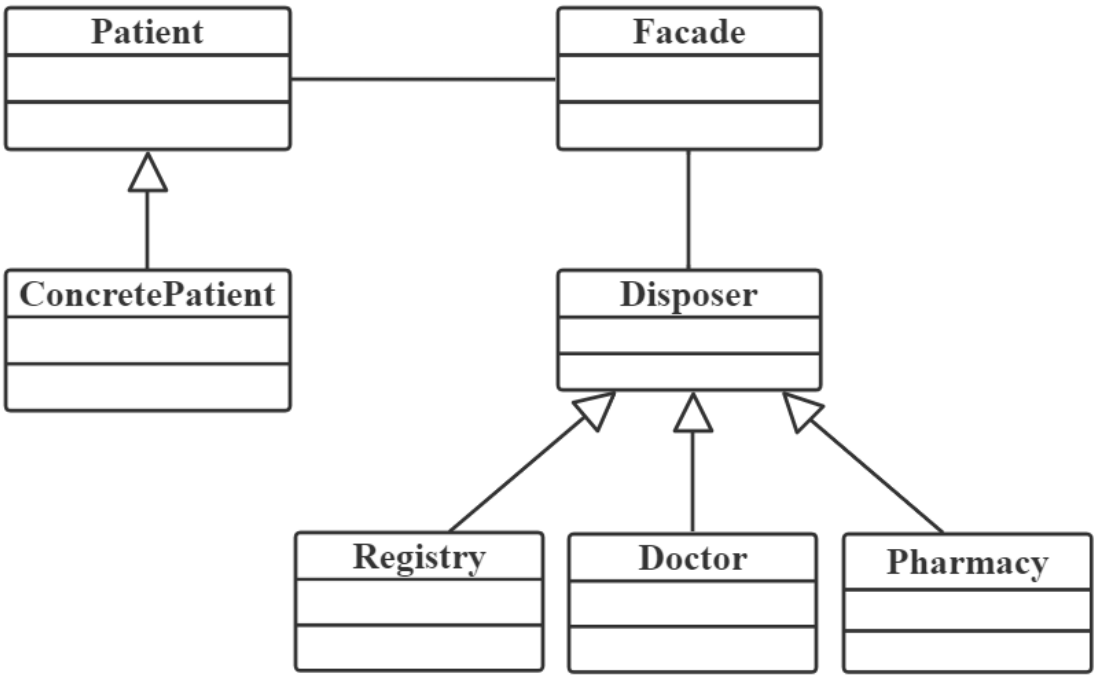


图 6-1 类图

【C++代码】

```
#include <iostream>
#include <string>

using namespace std;

class Patient {
public:
    __（1）__;
};
```

```
class Disposer {
public:
    （2）;
};

class Registry : public Disposer { // 挂号
public:
    void dispose(Patient *patient) {
        cout << "I am registering...." << patient->getName() << endl;
    }
};

class Doctor : public Disposer { // 医生门诊
public:
    void dispose(Patient *patient) {
        cout << "I am diagnosing...." << patient->getName() << endl;
    }
};

class Pharmacy : public Disposer { // 取药
public:
    void dispose(Patient *patient) {
        cout << "I am giving medicine...." << patient->getName() << endl;
    }
};
```

```
class Facade {
private:
    Patient *patient;

public:
    Facade(Patient *patient) { this->patient = patient; }

    void dispose() {
        Registry *registry = new Registry();
        Doctor *doctor = new Doctor();
        Pharmacy *pharmacy = new Pharmacy();

        registry->dispose(patient);
        doctor->dispose(patient);
        pharmacy->dispose(patient);
    }
};

class ConcretePatient : public Patient {
private:
    string name;

public:
    ConcretePatient(string name) { this->name = name; }

    string getName() { return name; }
};
```

```
int main() {
    Patient *patient = ___（3）___;
    ___（4）___ f = ___（5）___;
    ___（6）___;

    return 0;
}
```

全国计算机技术与软件专业技术资格（水平）考试

2023 年上半年 软件设计师 下午试卷

（考试时间 14:00~16:30 共 150 分钟）

请按下述要求正确填写答题卡

1. 在答题卡的指定位置上正确填写你的姓名和准考证号，并粘贴考生条形码。
2. 本试卷共 6 道题，试题一至试题四是必答题，试题五至试题六任选 1 道作答。每题 15 分，满分 75 分。
3. 解答时字迹务必清楚，字迹不清时，将不评分。
4. 仿照下面的例题，将解答写在答题纸的对应栏内。

（例题）

2023 年上半年全国计算机技术与软件专业技术资格（水平）考试日期是（1）月（2）日。

因为上半年考试的日期是“5 月 27 日”，故在答题纸的对应栏内写上“5”和“27”。（参看下表）

例题	解答栏
（1）	5
（2）	27

试题一至试题四为必答题

试题一（共 15 分）

阅读下列说明和图，回答问题 1 至问题 4，将解答填入答题纸的对应栏内。

【说明】

随着农业领域科学种植的发展，需要对农业基地及农事进行信息化管理，为租户和农户等人员提供种植相关服务。现欲开发农事管理服务平台，其主要功能是：

（1）人员管理。平台管理员管理租户；租户管理农户并为其分配负责的地块，租户和农户以人员类型区分。

（2）基地管理。租户填写基地名称、地域等描述信息，在显示的地图上绘制地块。

（3）种植管理。租户设定作物及其从种植到采收的整个农事过程，包括农事活动及其实施计划，农户根据相应农事过程提醒进行农事活动并记录。系统会在设定时间向农户进行农事提醒，对逾期未实施活动向租户发出逾期警告。

（4）投入品管理。租户统一维护化肥，杀虫剂等投入品信息。农户在农事活动中设定投入品的实际消耗。

（5）信息服务。用户按查询条件发起农事信息请求，对相关地块农事活动实施情况（如与农事过程比对）等农事信息进行筛选、对比和统计等处理，并将响应信息进行展示。系统也给其他第三方软件提供 APP 接口，通过接口访问的方式，提供账号，密码和查询条件发起农事信息请求，返回特定格式的农事信息，无查询条件时默返回账号下所有信息，多查询条件时返回满足全部条件的信息。

现采用结构化方法对农事管理服务平台进行分析与设计，获得如图 1-1 所示的上下文数据流图和图 1-2 所示的 0 层数据流图。

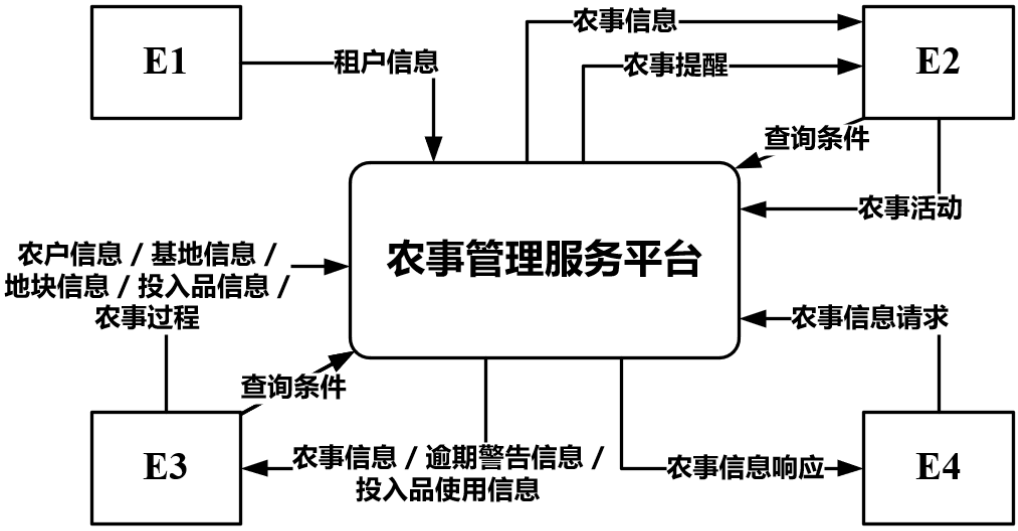


图1-1 上下文数据流图

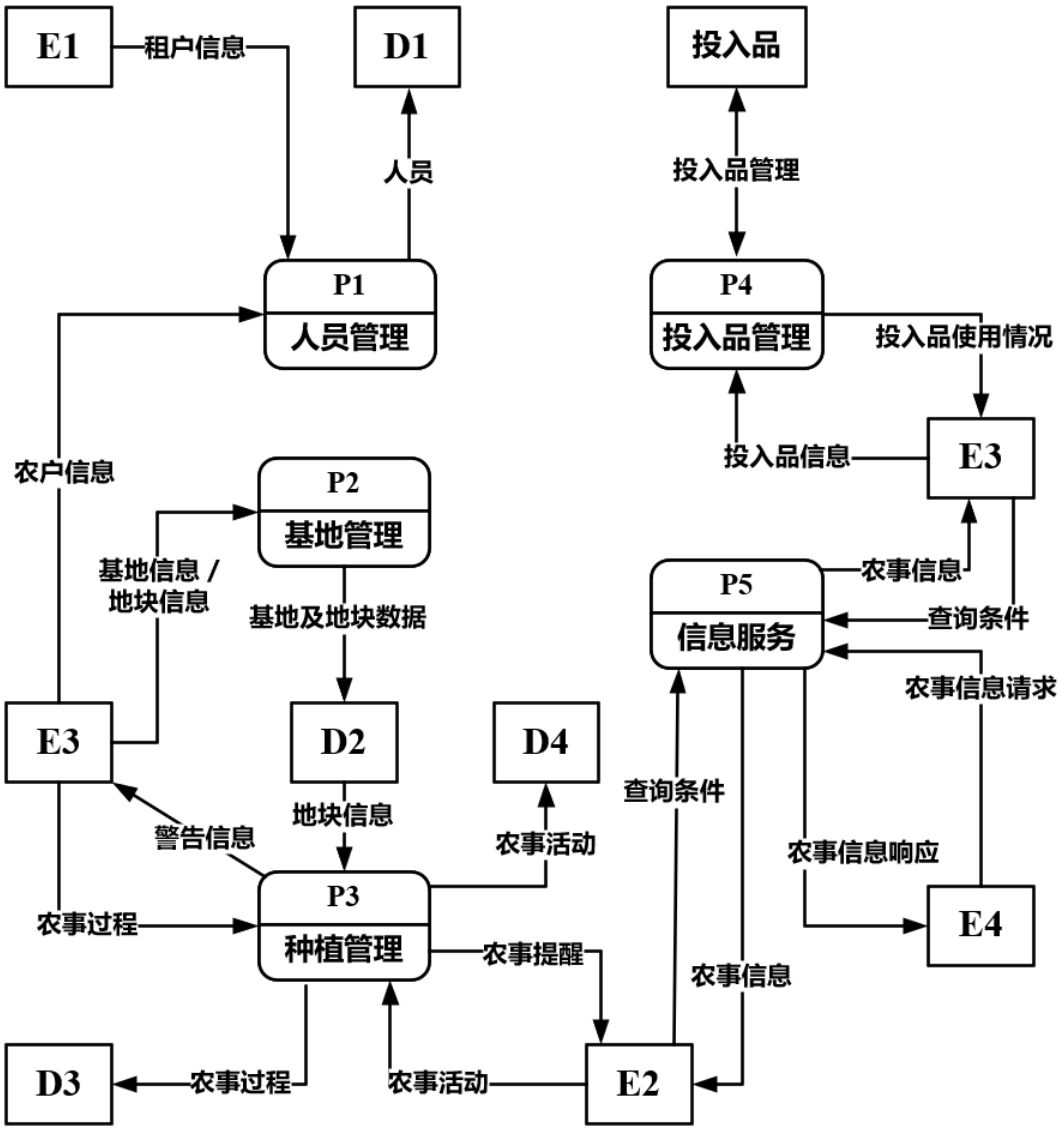


图1-2 0 层数据流图

【问题 1】（4 分）

使用说明中的词语，给出图 1-1 中的实体 E1~E4 的名称。

【问题 2】（4 分）

使用说明中的词语，给出图 1-2 中的数据存储 D1~D4 的名称。

【问题 3】（4 分）

根据说明和图中术语，补充图 1-2 中缺失的数据流及其起点和终点。

【问题 4】（3 分）

根据说明，给出“农事信息请求”数据流的组成。

试题二（共 15 分）

阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某新能源汽车公司为了提升效率。需开发一个汽车零件采购系统。请根据下述需求描述完成该系统的数据库设计。

【需求分析结果】

（1）记录供应商的信息，包括供应商的名称，地址和一个电话。

（2）记录零件的信息，包括零件的编码，名称和价格。

（3）记录车型信息，包括车型的编号，名称和规格。

（4）记录零件采购信息，某个车型的某种零件可以从多家供应商采购，某种零件也可以被多个车型采用，某家供应商也可以供应多种零件，还包括采购数量和采购日期。

【概念结构设计】

根据需求阶段收集的信息，设计的实体联系图（不完整）如图 2-1 所示。

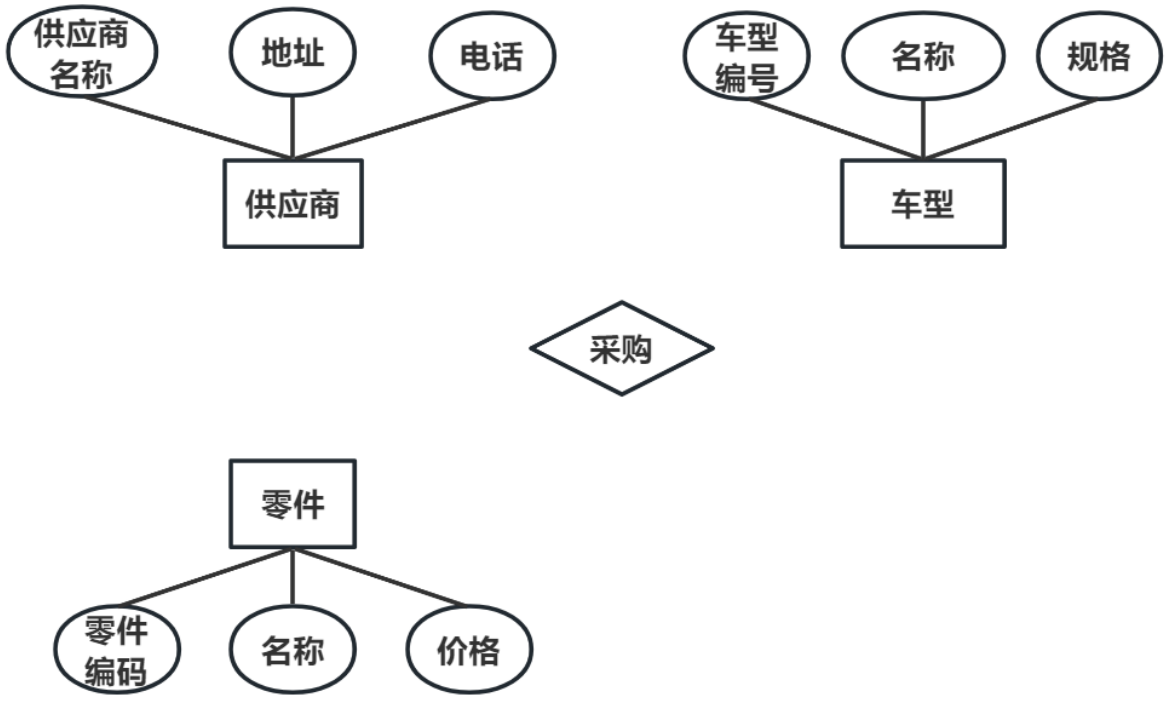


图 2-1 实体联系图

【逻辑结构设计】

根据概念模型设计阶段完成的实体联系图，得出如下关系模式（不完整）：

供应商（供应商名称，地址，电话）

零件（零件编码，名称，价格）

车型（车型编号，名称，规格）

采购（车型编号，供应商名称，（a），（b），采购日期）

【问题 1】（5 分）

根据需求描述，补充图 2-1 的实体联系图（不增加新的实体）。

【问题 2】（3 分）

补充逻辑结构设计结果中的（a），（b）两处空缺，并标注主键和外键完整性约束。

【问题 3】（7 分）

该汽车公司现新增如下需求：记录车型在全国门店的销售情况，门店信息包括门店的编号，地址和电话，销售包括销售数量和销售日期等

对原有设计进行以下修改以实现该需求：

（1）在图 2-1 中体现门店信息及其车型销售情况，并标明新增的实体和联系及其主要属性。

（2）给出新增加的关系模式，并标注主键和外键完整性约束。

试题三（共 15 分）

阅读下列说明和 UML 图，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。

【说明】

某高校图书馆购买了若干学术资源的镜像数据库（MinorDBMino）资源，现要求开发一套数字图书馆（Digitallibrary）系统，面向校内用户（User）提供学术资源（Resoure）浏览，检索和下载服务。系统的主要功能描述如下：

（1）系统中存储了每个镜像数据库的基本信息，包括：数据库名称，访问地址，数据库属性以及数据库简介等信息，用户进入某个镜像数据库后，可以浏览检索以及下载其中的学术资源。

（2）学术资源包括会议论文（ConferencePaper）、期刑论文（JounalArticle）以及学位论文（Thesis）等；系统中存储了每个学术资源的题名、作者、发表时间、来源（哪个镜像数据库）、被引次数、下载次数等信息。对于会议论文，还需记录会议名称，召开时间以及召开地点；同一次会议的论文被收录在会议集（Proceeding）中。对于期刊论文，还需记录期刊名称，出版月份，期号以及主办单位；同一期号的论文被收录在一本期刊（Edition）中。对于学位论文，记录了学位类别（博士/硕士），毕业学校，专业及指导教师。会议集包含发表在该会议（在某个特定时间段，特定地点召开）上的所有文章。期刊的每一期在特定时间发行，其中包含若干篇文章。

（3）系统用户（User）包括在校学生（Student），教师（Teacher）以及其他在职人员（Staff）。用户使用学校的统一身份认证登录系统后，使用系统提供的各项服务。

（4）系统提供多种资源检索的方式，主要包括：按照资源的题名检索（SearchByTitle），按照作者名称检索（SearchByAuthor），按照来源检索（SearchBySource）等。

（5）用户可以下载资源，系统记录每个资源被下载的次数。

现采用面向对象分析与设计方法开发该系统，得到如图 3-1 所示的用例图以及图 3-2 所示的类图。

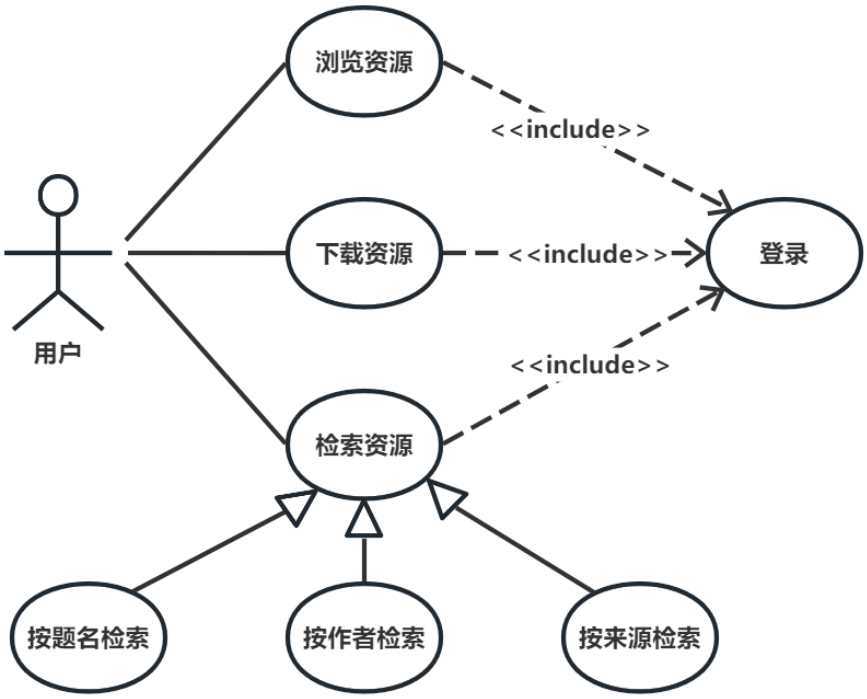


图 3-1 用例图

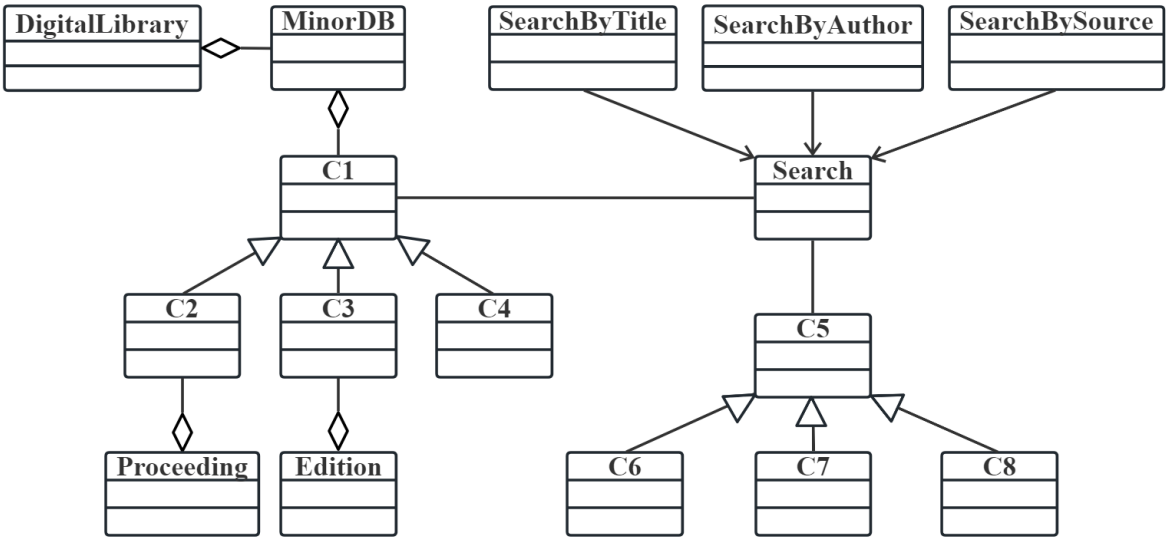


图 3-2 类图

【问题 1】（8 分）

根据说明中的描述，给出图 3-2 中 C1~C8 对应的类名。

【问题 2】（4 分）

根据说明中的描述，给出图 3-2 的类 C1~C4 的关键属性。

【问题 3】（3 分）

在该系统的开发过程中遇到了新的要求：用户能够在系统中对其所关注的数字资源注册他引通知，若该资源的他引次数发生变化，系候可以及时通知该用户，为了实现这个新的要求，可以在图 3-2 所示的类图中增加哪种设计模式？用 150 字以内文字解释选择该模式的原因。

试题四（共 15 分）

题目缺失

【C 代码】

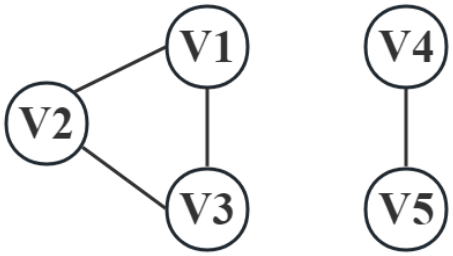
代码缺失

【问题 1】（10 分）

根据以上说明和 C 代码，填充 C 代码中的空（1）~（5）。

【问题 2】（5 分）

画出下图所示中无向图的邻接表。



从下列的 2 道试题（试题五至试题六）中任选 1 道解答。

请在答题纸上的指定位置将所选试题的题号框涂黑。

如多涂或者未涂题号框，则对题号最小的一道试题进行评分。

试题五 ☐　　**试题六** ☐

试题五（共 15 分）

阅读下列说明和 Java 代码，将应填入（n）处的字句写在答题纸的对应栏内。

【说明】

在某系统中，类 Interval 代表由下界（lower bound）和上界（upper bound）定义的区间。要求采用不同的格式显示区间范围。如：

[lower bound,upper bound];[lower bound...upper bound];[lower bound-upper bound]等。

现采用策略（Strategy）模式实现该要求，得到如图 5-1 所示的类图。

图 5-1 缺失

【Java 代码】

```
enum TYPE {
    COMMA,
    DOTS,
    LINE
}

interface PrintStrategy {
    public （1）;
}
```

```
class Interval {
    private double lower;
    private double upper;

    public Interval(double lower, double upper) {
        this.lower = lower;
        this.upper = upper;
    }

    public double getLower() { return lower; }

    public double getUpper() { return upper; }

    public void printIntervals(PrintStrategy ptr) {
        （2）;
    }
}

class PrintIntervalsComma implements PrintStrategy {
    public void doPrint(Interval val) {
        System.out.println("[" + val.getLower() + "," + val.getUpper() + "]");
    }
}

class PrintIntervalsDots implements PrintStrategy {
    public void doPrint(Interval val) {
        System.out.println("[" + val.getLower() + "..." + val.getUpper() + "]");
    }
}
```

```
class PrintIntervalsLine implements PrintStrategy {
    public void doPrint(Interval val) {
        System.out.println("[" + val.getLower() + "-" + val.getUpper() + "]");
    }
}

public class Main {
    public static PrintStrategy getStrategy(TYPE type) {
        PrintStrategy st = null;
        switch (type) {
            case COMMA:
                （3）;
                break;
            case DOTS:
                （4）;
                break;
            case LINE:
                （5）;
                break;
        }
        return st;
    }

    public static void main(String[] args) {
        Interval a = new Interval(1.7, 2.1);
        a.printIntervals(getStrategy(TYPE.COMMA));
        a.printIntervals(getStrategy(TYPE.DOTS));
        a.printIntervals(getStrategy(TYPE.LINE));
    }
}
```